1. Introduction

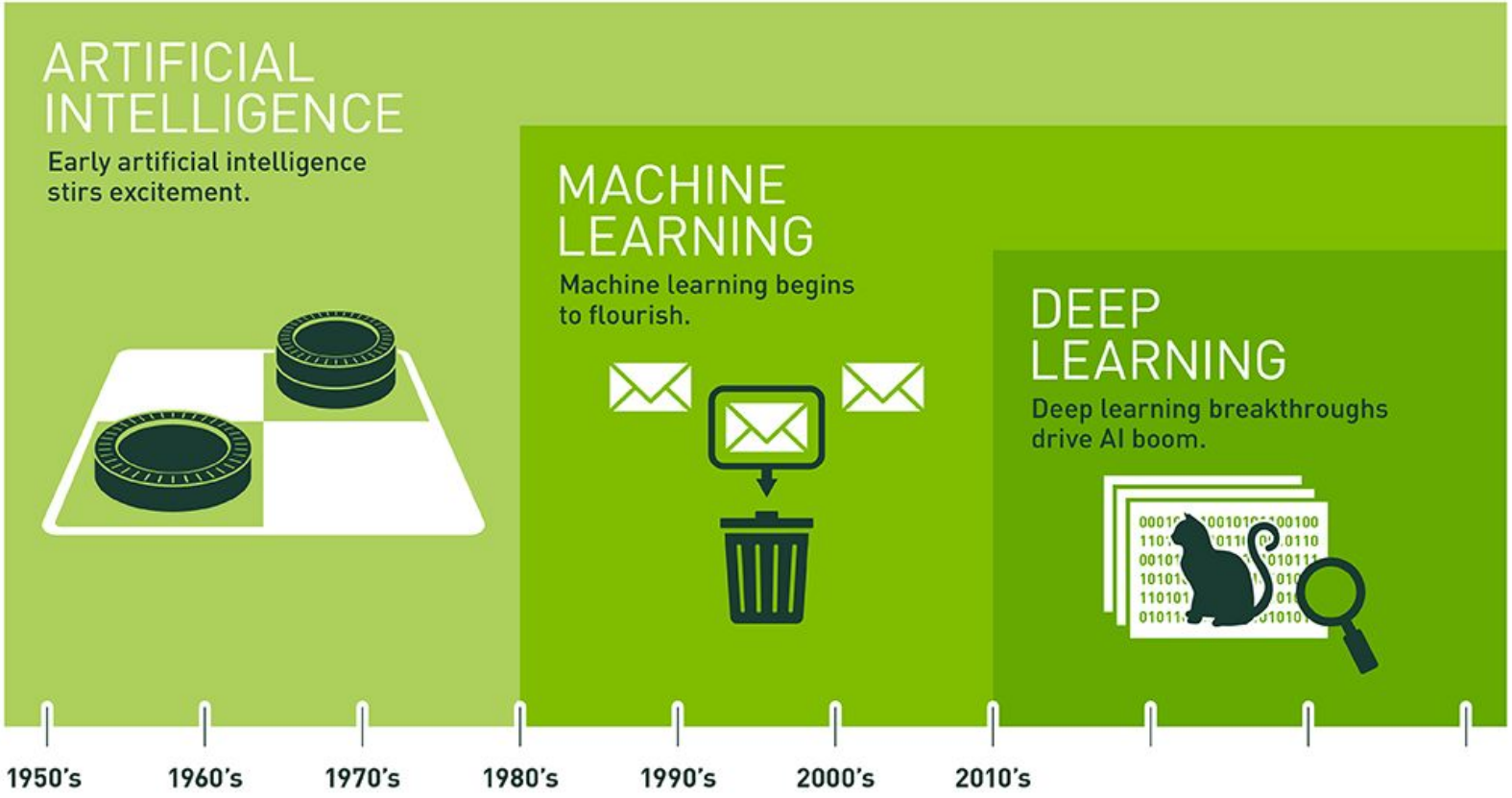
Machine learning

Doc. Ing. Radim Kolar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Outline

1. Basic terminology
2. Data types
3. Performance of classifiers
4. Performance of regression

1. Basic terminology

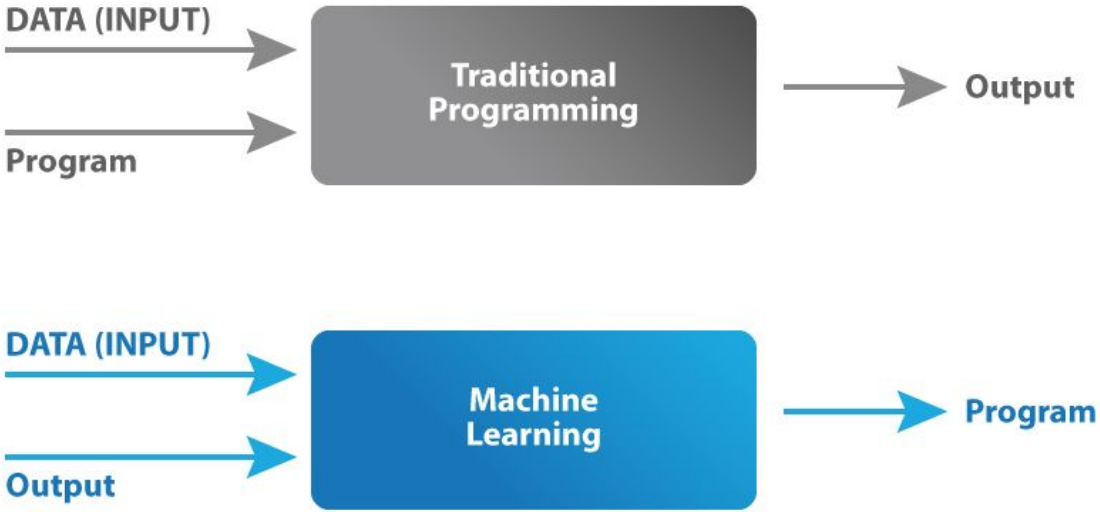


Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

https://www.datasciencecentral.com/profiles/blogs/artificial-intelligence-vs-machine-learning-vs-deep-learning

ML definition

The term Machine Learning was coined by Arthur Samuel in 1959, an American pioneer in the field of computer gaming and artificial intelligence and stated that “it gives computers the ability to learn without being explicitly programmed”.



https://d1m75rqqgidzqn.cloudfront.net/2019/10/What-is-Machine-Learning-Machine-learning-model-vs-traditional-model.jpg

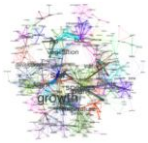
Recent ML definition(s)

“Machine Learning at its most basic is the practice of using algorithms to parse data, learn from it, and then make a determination or prediction about something in the world.” **– Nvidia**

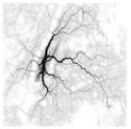
“Machine Learning is the science of getting computers to learn as well as humans do or better.” **- Dr. Roman Yampolskiy, University of Louisville**

“Machine Learning is the science of getting computers to learn and act like humans do, and improve their learning over time in autonomous fashion, by feeding them data and information in the form of observations and real-world interactions.” - **https://emerj.com/ai-glossary-terms/what-is-machine-learning/**

https://btimmermans.com/2017/12/11/machine-learning-overview



Structure Discovery

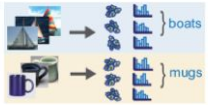


Big Data Visualisation

Machine Learning

Unsupervised Learning

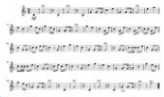
Dimension Reduction



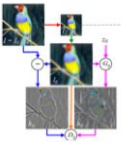
Feature Extraction

Generative Networks

2D to 3D Modelling



Music Generation



Pattern Modelling

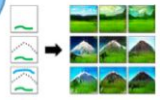
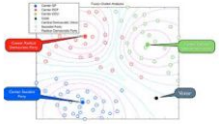
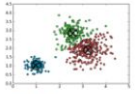


Image Generation

Clustering



Recommender Systems



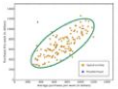
Customer Segmentation

Targeted Marketing

Supervised Learning

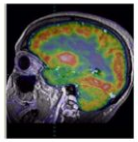
Classification

Image Classification



Fraud Detection

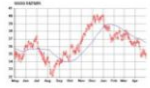
Gender Detection



Medical Diagnosis

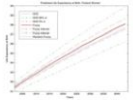
Weather Forecasting

Regression

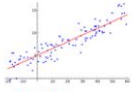


Sales Growth Prediction

Advertising Reach Prediction



Estimating Life Expectancy



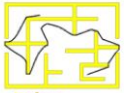
Market Forecasting

Reinforcement Learning

Real-time Decisions

Game AI

Skill Acquisition

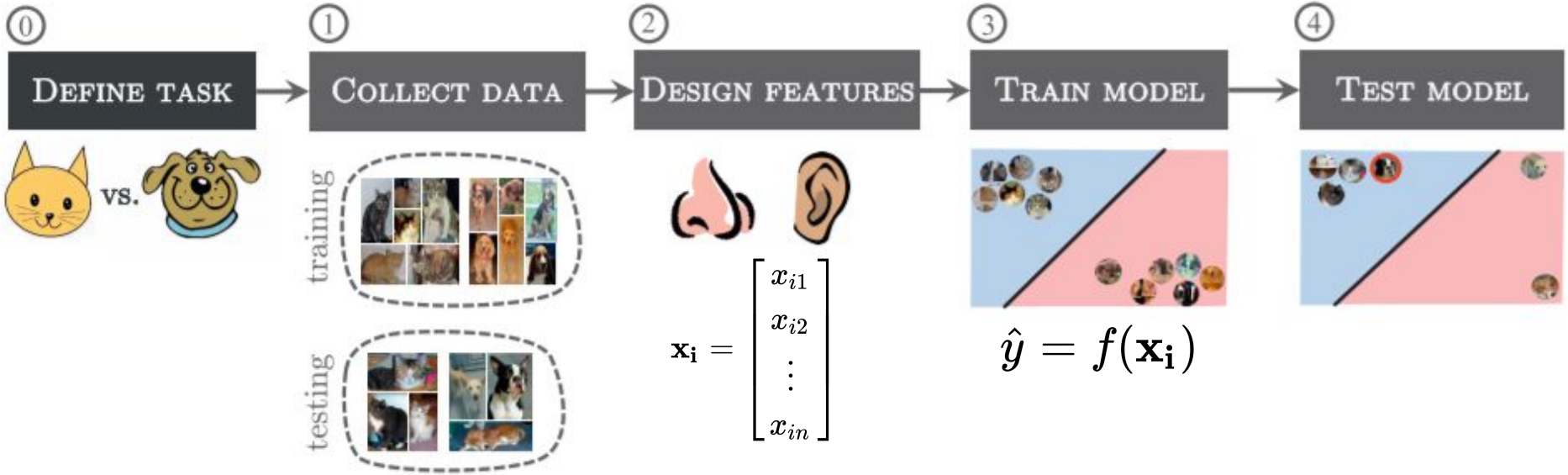


Robot Navigation

Learning Tasks

ML introduction

A typical (now, we can also say traditional) machine learning pipeline can be split into several steps.



Wat, Borhani, Katsagfelos: Machine learning refined, Cambridge University Press, 2019

The typical machine learning pipeline

0. **Define the problem** - what is the task we want to teach a computer to do? Detection of objects in images? Classification? Regression?

1. **Collect the data** for training and evaluation - the larger and more diverse the data the better.

2. **Design the features** - what features describe best our data? Decide if some feature selection or reduction is needed. The features then forms a feature vector $\mathbf{x}_i$ for one *measurement* (with possible label y_i), for i=1,...n.

3. **Select, set and train the model** - this model takes the feature vector and gives an outcome (output values): $\hat{y} = f(\mathbf{x_i})$

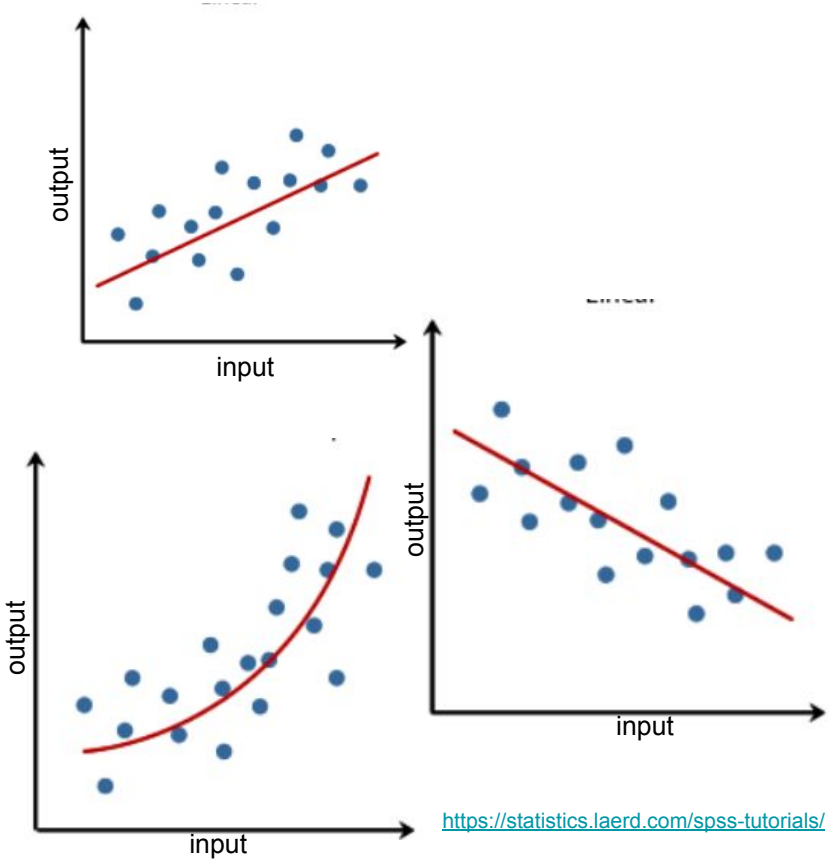
4. **Test the model** - evaluate the performance of the trained model. Tune the parameters of the model.

Regression

Predictive learning problems constitute the majority of tasks. There can be identified two major categories: Regression and Classification

Regression - the main output of the model is a function *f* that automatically produces a prediction, denoted with $\hat{y}$, for a vector of predictors (i.e. feature samples): $\hat{y} = f(\mathbf{x})$

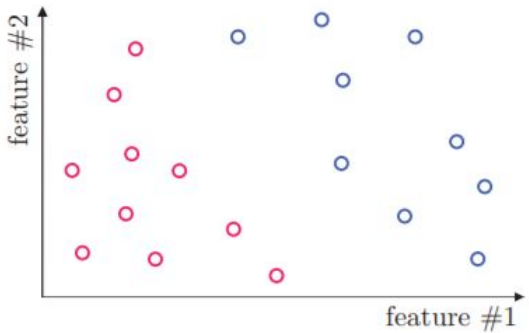
So we want this prediction to match the actual outcome as best as possible. Because the outcome is continuous, our predictions will not be either exactly right or wrong, but instead we will determine an error defined as the difference between the prediction and the actual outcome: $|\hat{y} - y|$

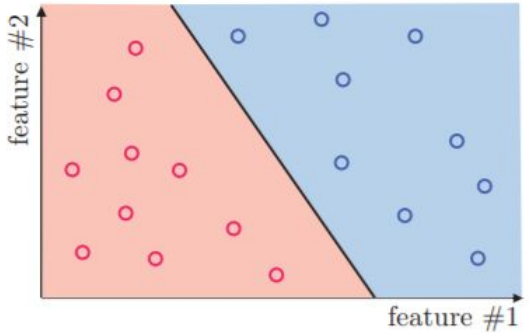


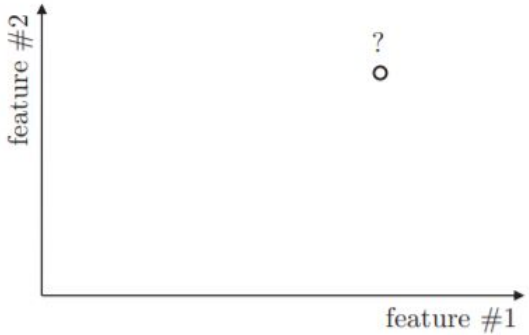
https://statistics.laerd.com/spss-tutorials/

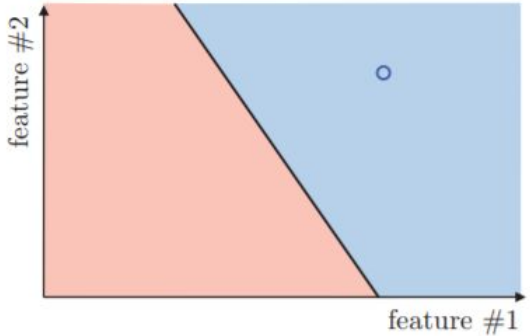
Classification

In classification the main output of the model will be a decision rule which prescribes which of the *K* classes (i.e. discrete values; there can be from *2* to *N* classes) we should predict based on input feature vector **x**. In this scenario, most models provide functions of the predictors for each class $f_k(\mathbf{x})$, that are used to make this decision.









Multiclass and multilabel classification

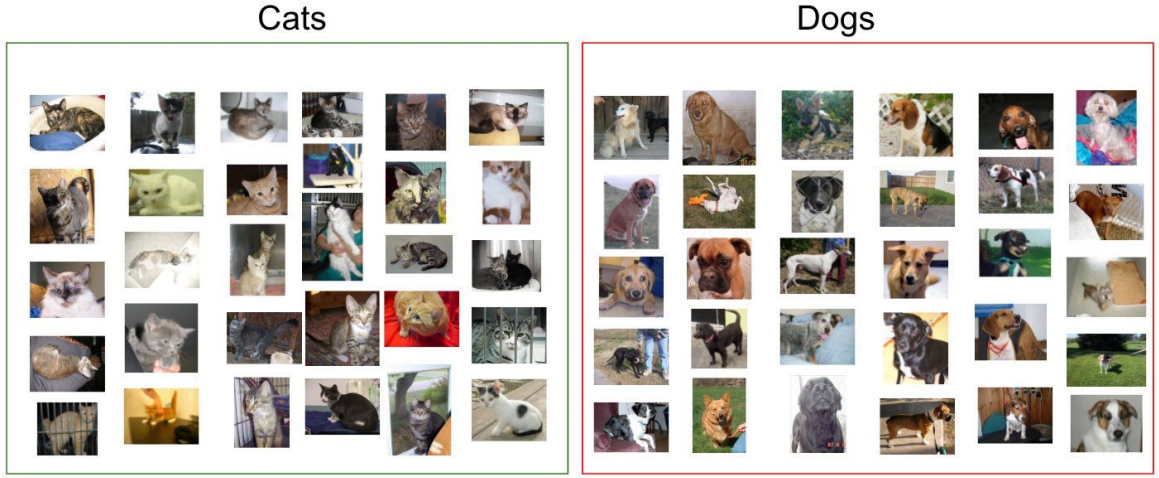
- **multiclass** (K > 2); e.g. classification of objects in the image (car, bus, train, bike...)




- **multilabel** - one input can belong to more classes;
  e.g.the image can contain different objects (car and bike)


Supervised learning

The machine has a "supervisor" or a "teacher" who gives the machine all the answers, like whether it's a cat in the picture or a dog. The “teacher” has already divided (labeled) the data into cats and dogs, and the machine is using these examples to learn.



Sample of cats & dogs images from Kaggle Dataset

Supervised learning

Data: ($\mathbf{x}_i$,y_i)

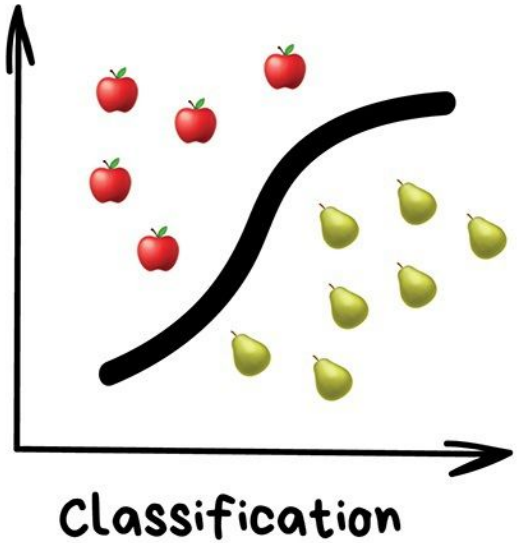
x is data, usually in the form of vectors of features i.e.

$$\mathbf{x_i} = \begin{bmatrix} x_{i1} \\ x_{i2} \\ . \\ . \\ x_{in} \end{bmatrix}$$

y represents labels of classes, for example y_i=+1 or -1 for binary case.

Goal: Learn a function to map x to y

Example: classification

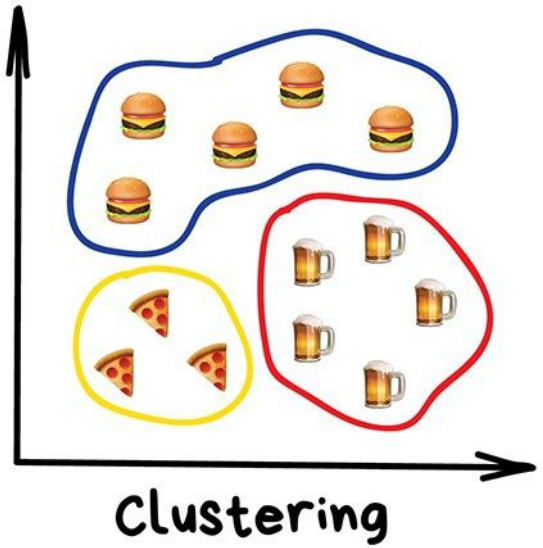


Unsupervised learning

Data: Just data, no labels.

Goal: Learn some *hidden* structure of the data.

Example: Clustering.



Reinforcement learning

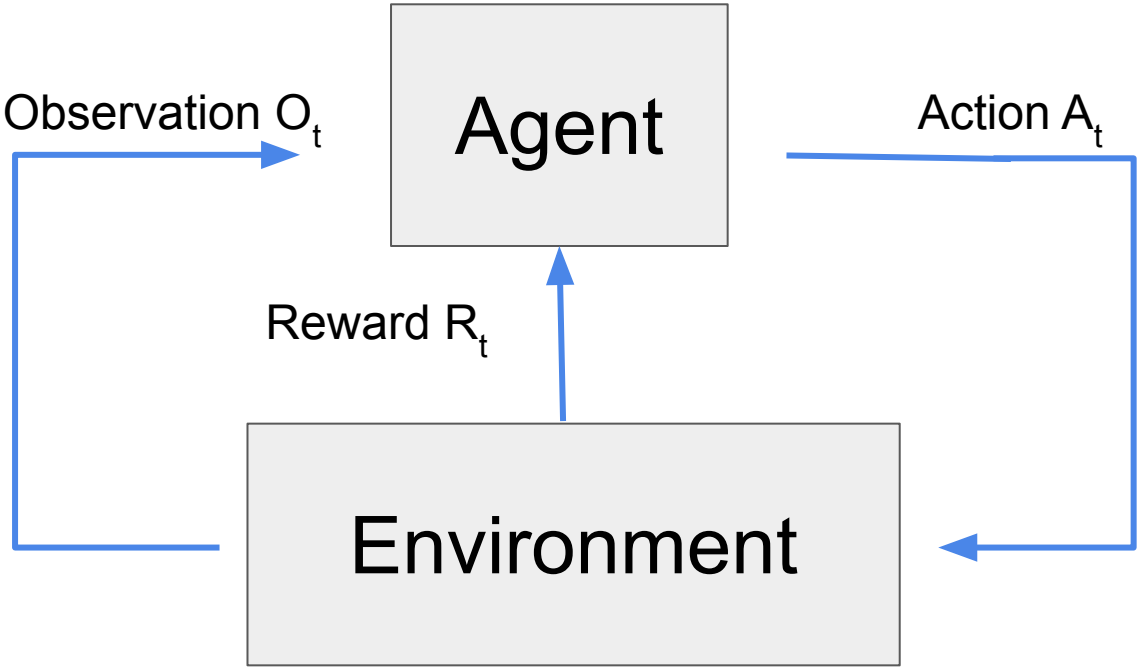
Skip this

Reinforced learning uses a straightforward approach - the action takes place, the consequences are observed, and the next action considers the results of the first action.

So, it is about taking suitable **action** to maximize **reward** in a particular situation. It is employed by various software and machines to find the best possible behavior or path it should take in a specific situation. Reinforcement learning differs from the supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task.

Reinforcement learning

Skip this



<u>At each step the agent:</u>
- executes action A_t
- receives observation O_t
- receives scalar rewards R_t

<u>The environment:</u>
- receives action A_t
- emits observation O_t
- emits scalar rewards R_t

Goal: select actions to maximize total future reward

Remarks: Actions may have long term consequences; Reward may be delayed; It may be better to sacrifice immediate reward to gain more long-term reward.

Reinforcement learning

Skip this

Examples:

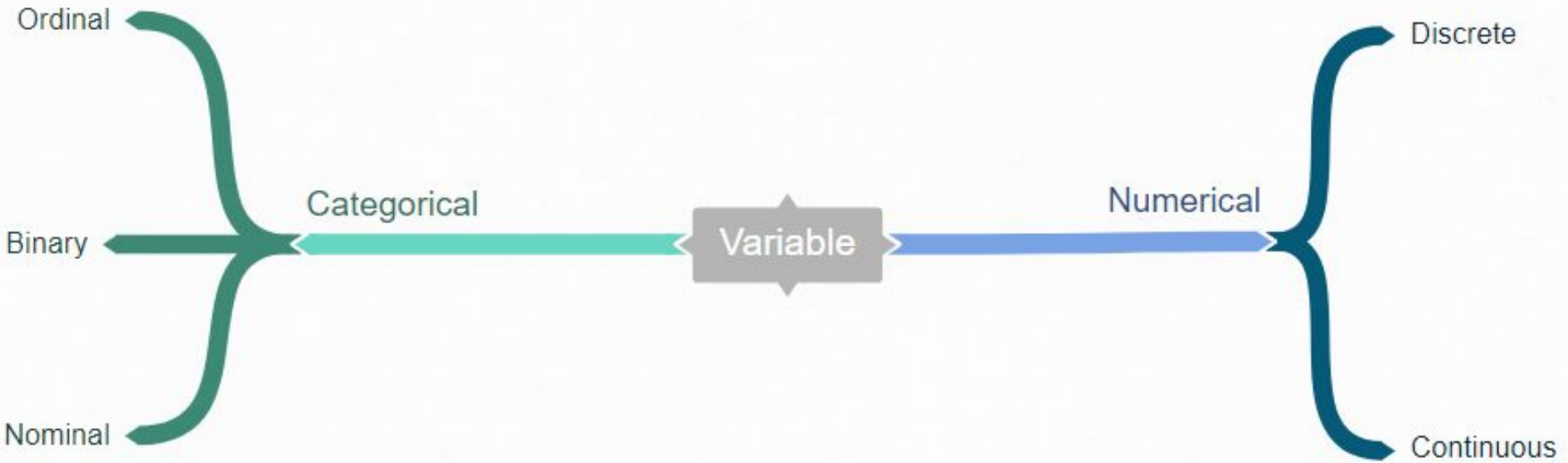
How DeepMind Conquered Go With Deep Learning (AlphaGo):
https://www.youtube.com/watch?v=IFmj5M5Q5jg

Self-driving car:
https://www.youtube.com/watch?time_continue=129&v=eRwTbRtnT1I

Stock prediction:
https://www.youtube.com/watch?v=05NqKJ0v7EE

2. Data types

Data/variable types



Data types

Numerical data is information that is measurable, and it is data represented as numbers and not words or text. This data can be **continuous** numbers (e.g. weight, temperature) and **discrete** numbers (e.g. number of days, greylevels).

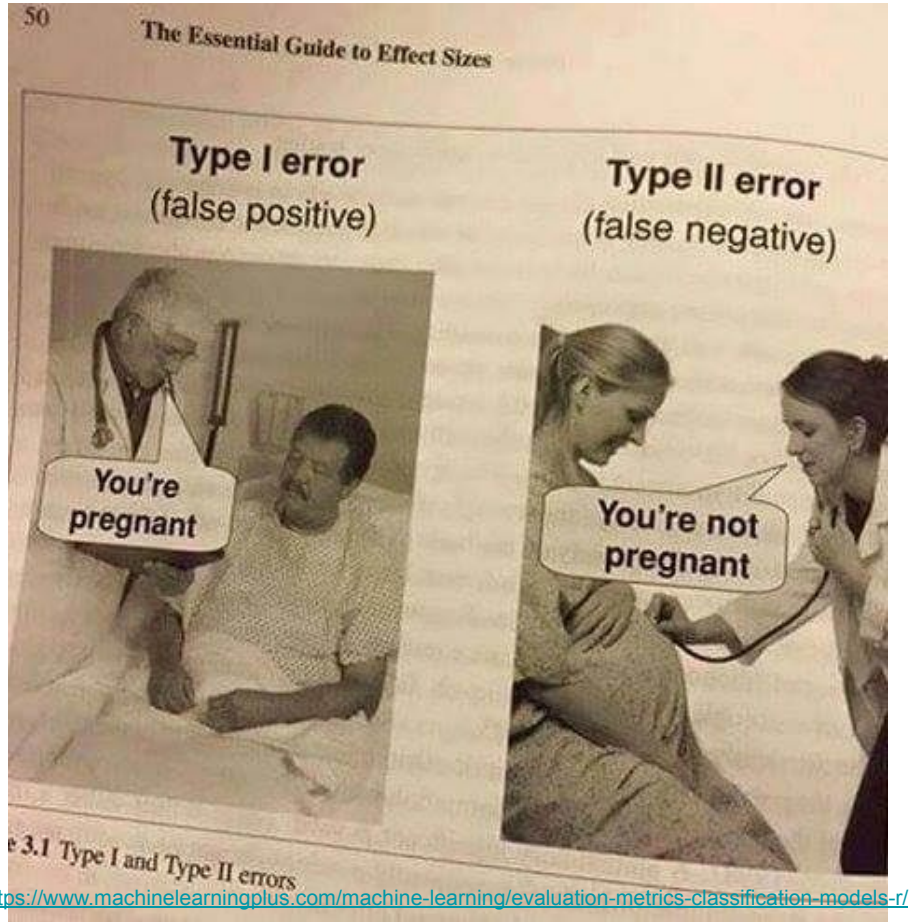
Categorical data can be split to nominal, ordinal or binary.

Nominal variable is one that has two or more categories, but there is no intrinsic ordering to the categories. For example hair color is a nominal variable having several categories.

An **ordinal variable** is similar to a nominal variable. The difference between the two is that there is a clear ordering of the variables. For example, suppose variable - *economic status*, with three categories (low, medium and high). We can classify people into these three categories.

Binary variable can have only two states - for example, *gender* is a binary variable having two categories (male and female). Also named as **dichotomous variable**.

3. Classifier performance



https://www.machinelearningplus.com/machine-learning/evaluation-metrics-classification-models-r/

Why we need to test classifier performance?

1. Get a reliable estimate of the classifier performance – i.e. how it will perform on new – so far unseen – data instances.
2. Compare your classifiers that you have developed – to decide which one is “the best”.

This helps you with model assessment and selection.

Remark: Whenever you extend your training data, you should get a better classifier! If not, something is wrong :((not representative data, wrong model, setting of learning parameters).

Main strategies

Classifier performance evaluation

Classifiers are learned (**trained**) on a finite training dataset multiset (named simply a training set).
A learned classifier has to be **tested on a different test** set in order to evaluate the performance.
Usual way how to train and test the classifier is to divide the dataset into training and testing part. But there is some trade-off:

1. More training data gives better generalization.
2. More test data gives better estimate for the classification error.

Caution: **Never evaluate performance on training data!** The conclusion would be optimistically biased.

Classifier performance evaluation

Partitioning of available finite set of data to training / test sets. Strategy of partitioning:

1. Hold out.
2. **Cross validation.**

Once evaluation is finished, all the available data can be used to train the final classifier.

Hold out method

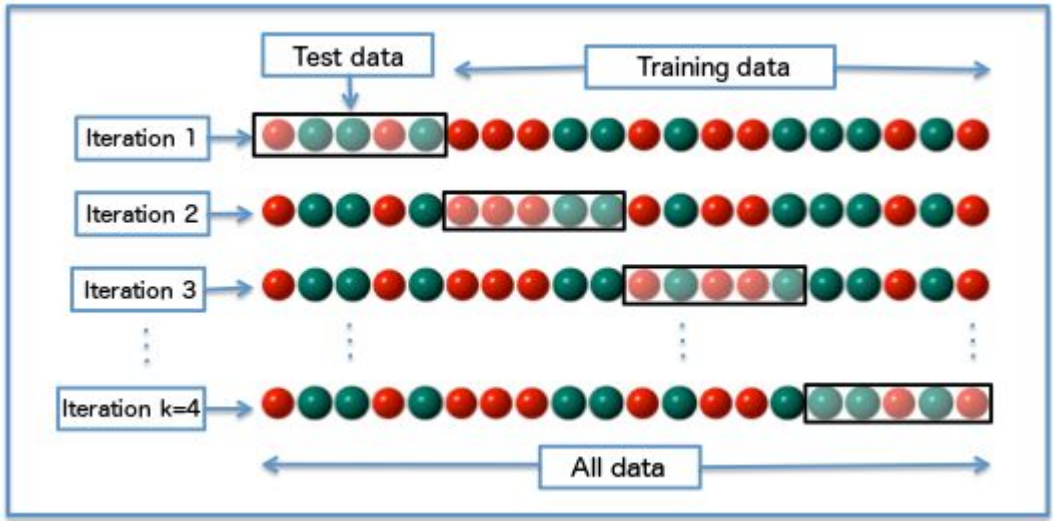
Given data is **randomly** partitioned into two independent sets.

1. Training set (e.g., 2/3 of data) for learning the classifier.
2. Test set (e.g., 1/3 of data) is hold out for the accuracy estimation of the classifier.

Then: Repeat this random **hold out** *k* times; the accuracy is estimated as the average of the accuracies obtained during these two steps.

K-fold cross validation

1. The training set is randomly divided into *K* disjoint sets of equal size where each part has roughly the same class distribution (the same number of samples in each class).
2. The classifier is trained *K* times, each time with a different set held out as a test set. Therefore, we have K estimates of train error.
3. The estimated error is the mean of these *K* errors. The standard deviation of error is also important.



https://en.wikipedia.org/wiki/Cross-validation_(statistics)

Recommended experimental validation procedure

1. Use K-fold cross-validation (K is between 5 to 20) for estimating performance estimates (accuracy, etc.).
2. Compute the mean value of performance estimate, and standard deviation.
3. Report mean values of performance estimates and their standard deviations or 95% confidence intervals around the mean.

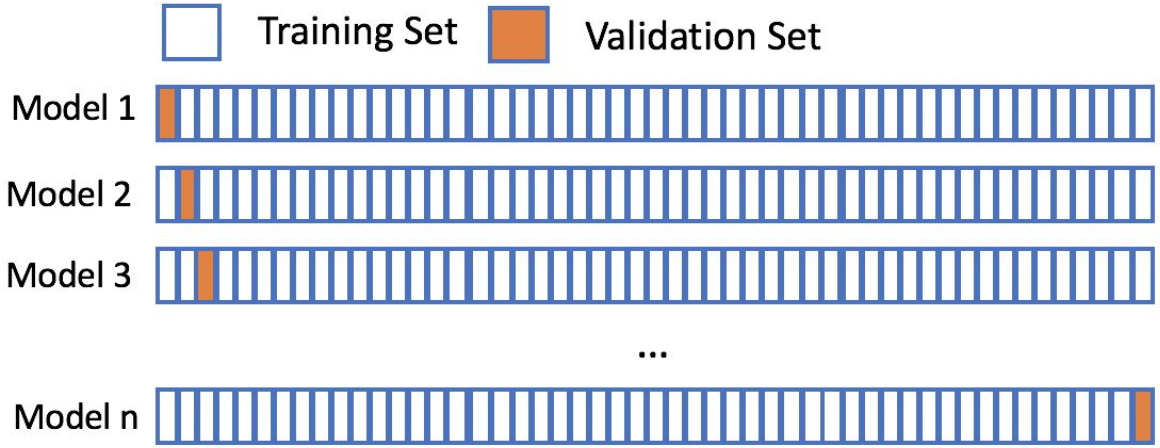
Remark:
Although the cross-validation is usually connected with classifiers, it is also used for evaluation of regression model.

Leave-one-out cross validation

1. A special case of K-fold cross validation with K = n, where n is the total number of samples in the training multiset.
2. Totally n experiments are performed using n − 1 samples for training and the remaining sample for testing.

It is rather computationally expensive. Image that you have e.g. 10000 samples.

Advantage: Uses maximum training set.



Unbalanced dataset

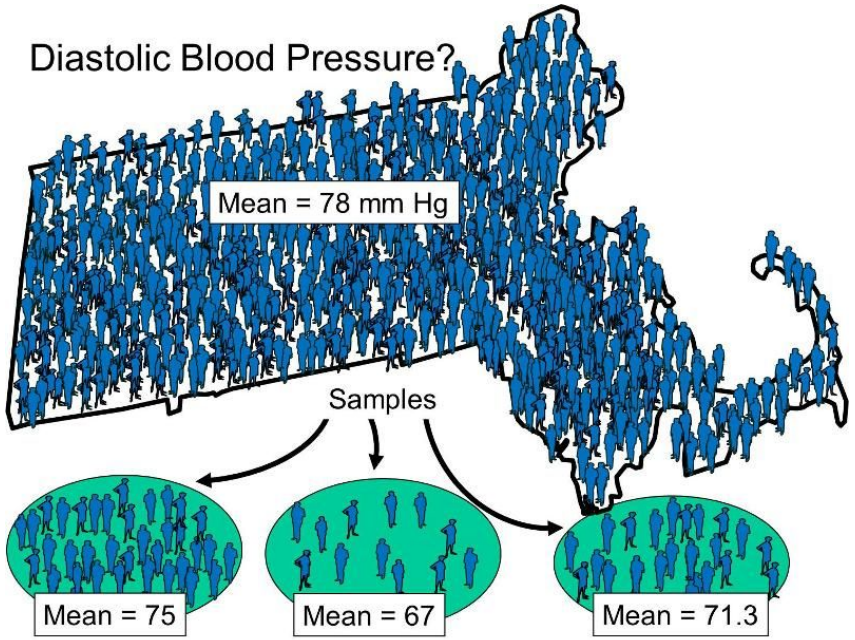
Unbalanced problems

Classes have often unequal frequency and it is difficult to have the same number of data/samples/examples in both (all) classes.

- Medical diagnosis: 95 % healthy, 5% disease.
- Security: 99.999 % of citizens are not terrorists.

The problem arises:
How should we train classifiers and evaluated them for unbalanced problems?

Remark about data sampling



A **population** is the collection of all subjects of interest.
A **sample** is a subset of the population of interest.

https://sphweb.bumc.bu.edu/otlt/MPH-Modules/BS/BS704_BiostatisticsBasics/BS704_BiostatisticsBasics2.html

Unbalanced problems

Usually two main approaches are used:

1. **undersampling** - removing majority examples
2. **oversampling** - replicates the minority examples

Researchers/Data analysts often use oversampling procedures since they are capable of balancing class distributions without ruling out potentially critical majority examples.

Example - undersampling

Consider two-class problem. Our dataset contains one class (A,B,C,D,E) and second class (1,2,3,4,5,6,7,8,9,10). Therefore the unbalance is 50:100.

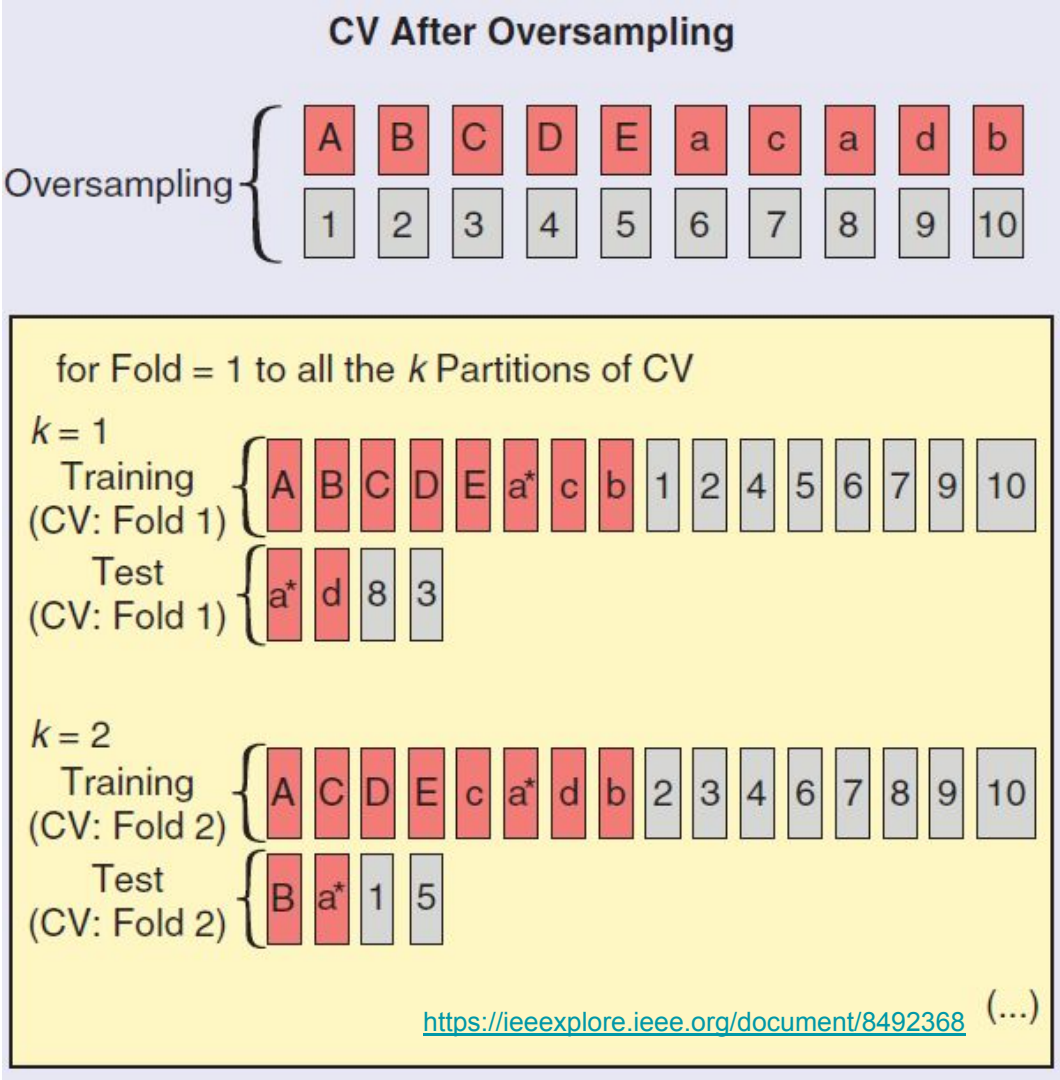
Undersampling means that we select and throw away set of samples from the second class to make the classes balanced, for example (1, 5, 7, 8, 10). This can be dangerous because we can throw away “important” samples.

https://ieeexplore.ieee.org/document/8492368

(Wrong) Example - oversampling 😞

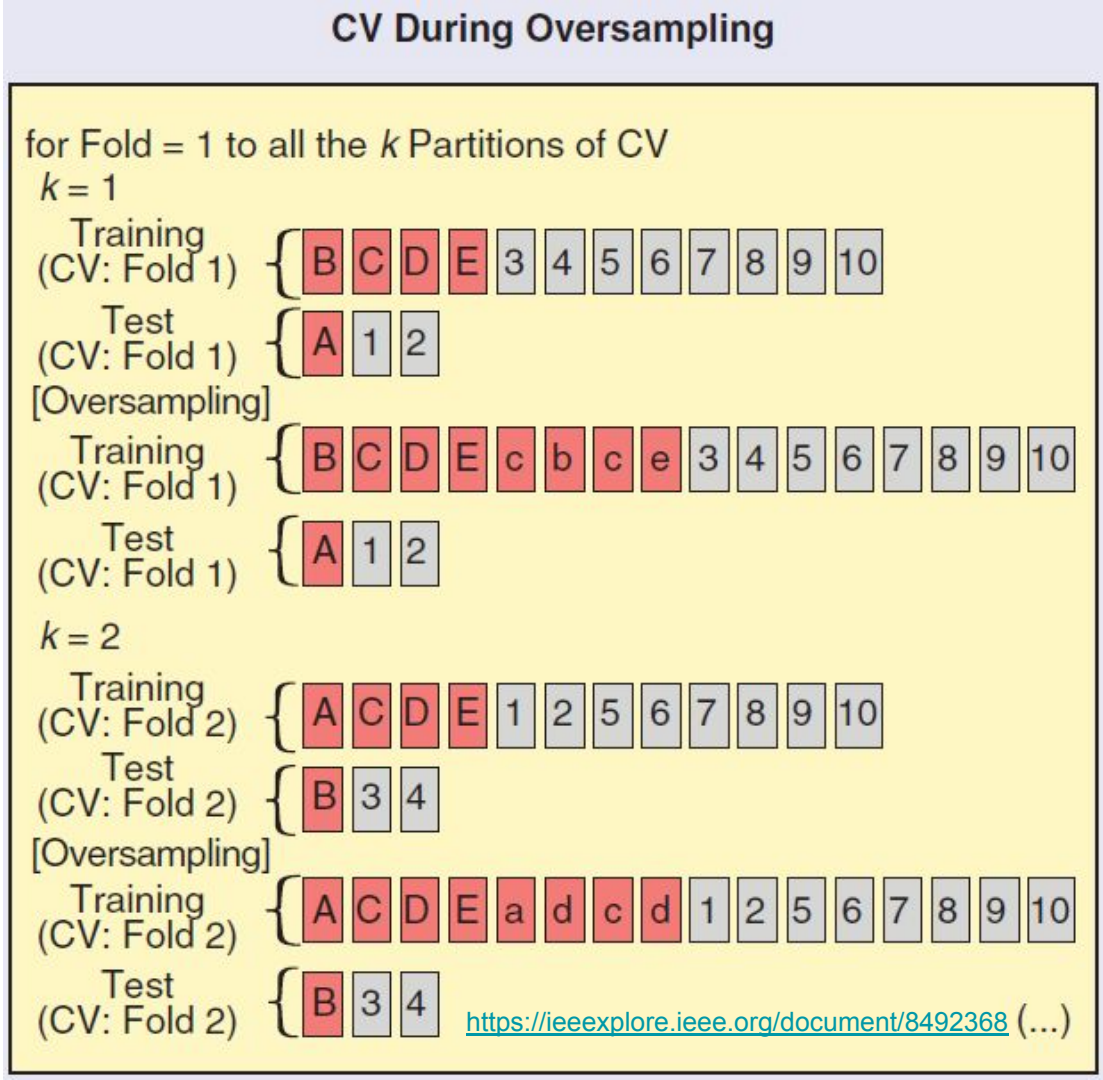
Consider the same 2-class example. Now, perform oversampling first and then do the cross-validation (CV).

During oversampling, one or several copies of some sample can appear in the new (oversampled) dataset (depending on the level of unbalance and strategy of sample selection). Consequently, it is possible that copies of the same patterns appear in **both the training and test sets**, making this design subjected to ***overoptimism***.



(Correct) Example - oversampling 😊

In the second approach, the oversampling procedure is performed during cross-validation: the dataset is first divided into k stratified partitions and only the training set (corresponding to k - 1 partitions) is oversampled. In this scenario, the patterns included in the test set are never oversampled or seen by the model in the training stage, thus allowing a proper evaluation of the model's capability to generalize from the training data.



Overoptimism

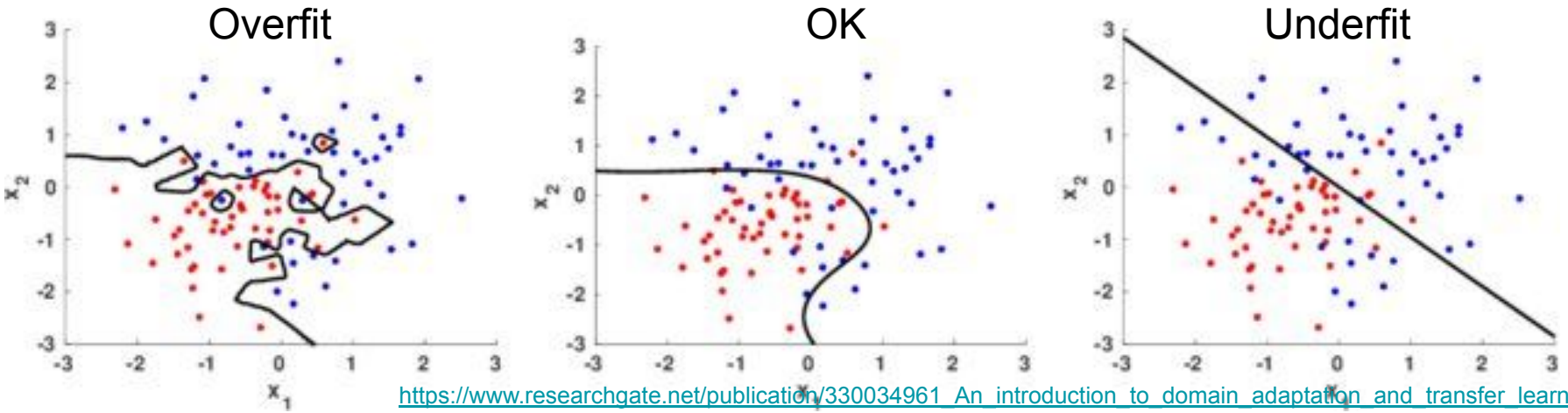
Overoptimism occurs when exact or similar replicas of a given pattern exist in both the training and test sets. In this case, the classification performance in the test sets will be similar to the one obtained in the training sets, not because the model is able to correctly generalize to the test data, but rather because there are similar patterns in both training and test partitions.

In this context, overoptimism is associated to incorrect implementations of cross-validation approaches, when oversampling is used.

Overfitting, underfitting

Overfitting occurs when the classifier is “tightly fitted” to the training data points, and therefore loses its generalization ability for the test data. Because of this, the classification performance is lower in the test set when compared to the training set.

Underfitting - the classifier if not learned enough.



https://www.researchgate.net/publication/330034961_An_introduction_to_domain_adaptation_and_transfer_learning

Oversampling - ROS

There are many methods for oversampling. Review is for example here: Santos et al. https://ieeexplore.ieee.org/document/8492368

Two main approaches:

1. **ROS** - Random Oversampling - is the simplest of oversampling techniques, where the existing minority examples are replicated until the class distribution is balanced. This approach is often criticised since it does not introduce any new information to the data (the oversampled examples are mere copies of the original data points) and may lead to overfitting (even if cross-validation is performed properly).

Oversampling - SMOTE

2. **SMOTE** - Synthetic Majority Oversampling Technique - works by generating synthetic minority examples along the line segments joining randomly chosen *G* minority examples and their *k*-nearest minority class neighbors. *G* is the number of minority examples to oversample in order to obtain the desired balancing ratio between the classes, and along with the value of *k*, it can be specified by the user.

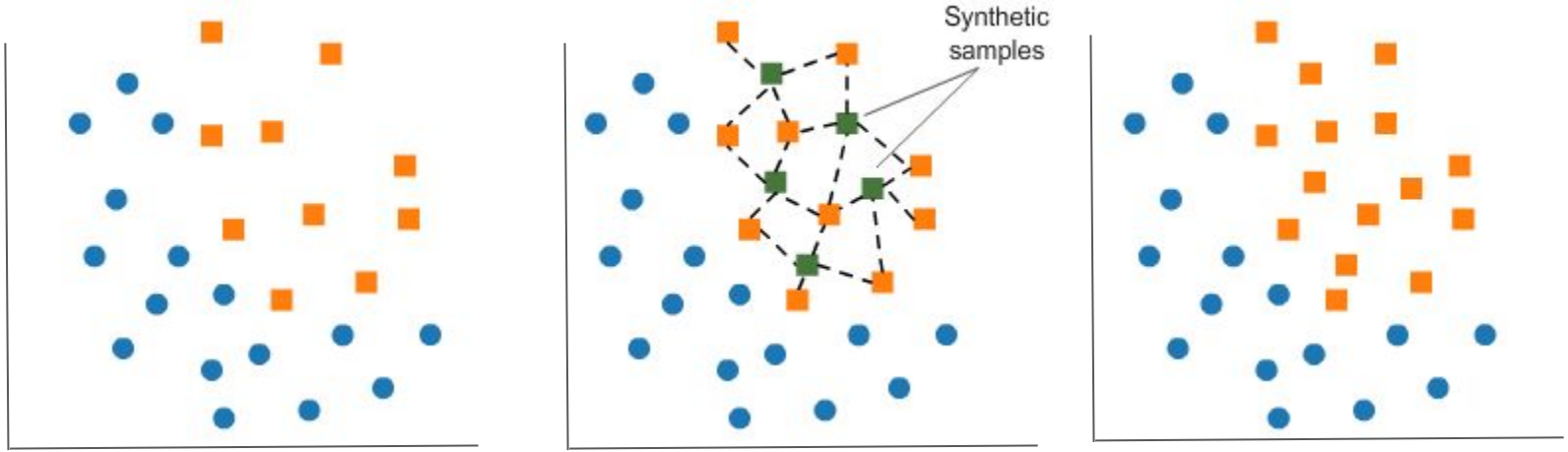


Image from: https://www.kaggle.com/rafjaa/resampling-strategies-for-imbalanced-datasets

Oversampling - SMOTE

SMOTE will then **generate a new synthetic sample *s*** according to $\boldsymbol{s = x + r(x - v)}$, where **x** is the minority sample to oversample, **v** is one of its chosen nearest neighbors and *r* is called a gap, in this case, a random number between 0 and 1. By generating similar examples to the existing minority points, SMOTE creates larger and less specific decision boundaries that increase the generalization capabilities of classifiers, therefore increasing their performance.

Classifier performance assessment

Criterion function to assess classifier performance

Typically, two quantities are evaluated:

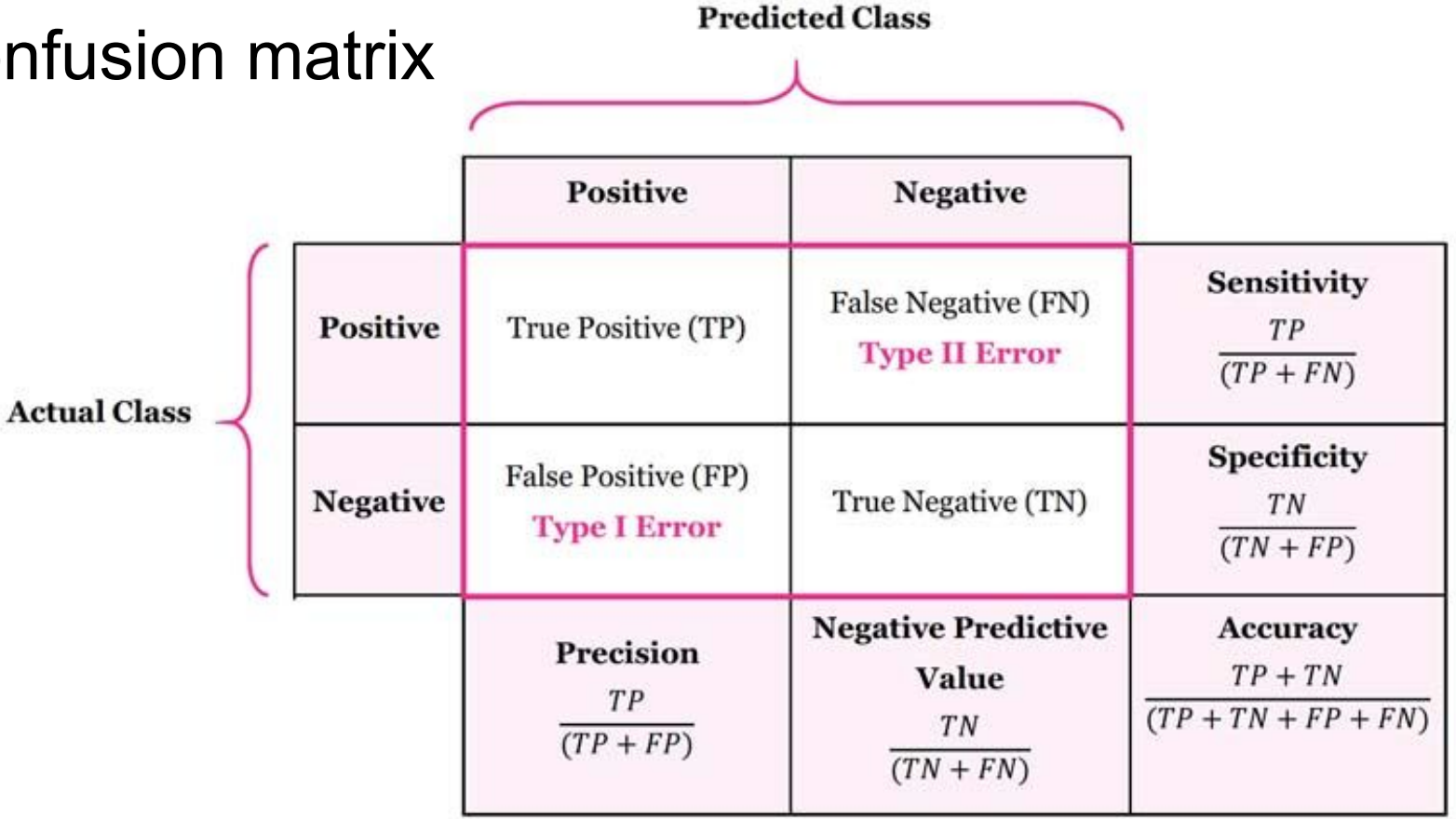
- **Accuracy** is the percent of correct classifications.
- **Error rate** = is the percent of incorrect classifications.

It’s obvious that:

$$\text{Accuracy} = 1 - \text{Error rate}$$

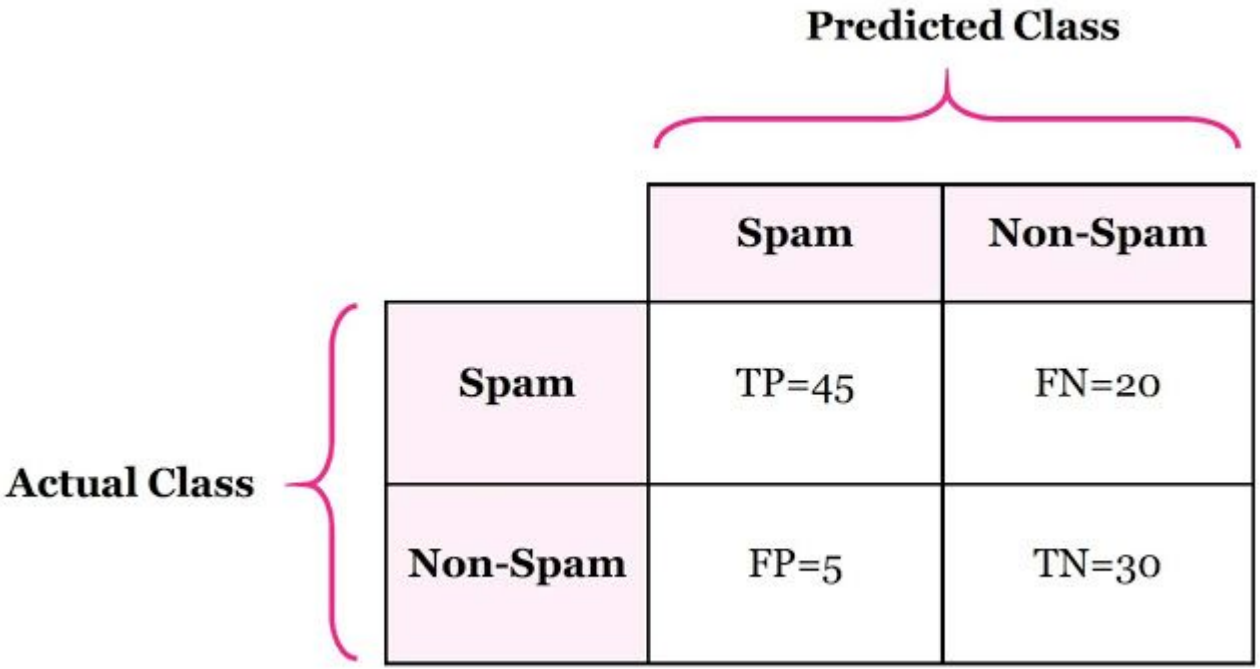
Other characteristics derived from the **confusion matrix**.

Confusion matrix



https://manisha-sirsat.blogspot.com/2019/04/confusion-matrix.html

Confusion matrix - example



Confusion matrix - multiclass, example

▾ TRAINING METRICS - CONFUSION MATRIX

	0	1	2	3	4	5	6	7	8	9	Error	Rate
0	902	0	10	5	1	12	3	3	7	3	0.0465	44 / 946
1	0	1057	8	4	2	6	4	6	14	7	0.0460	51 / 1,108
2	14	11	826	25	23	5	17	17	25	4	0.1458	141 / 967
3	7	6	15	900	2	39	2	16	33	11	0.1271	131 / 1,031
4	1	3	13	1	893	3	5	7	3	52	0.0897	88 / 981
5	14	7	7	25	13	814	29	3	26	15	0.1459	139 / 953
6	8	5	25	1	21	18	875	3	7	4	0.0951	92 / 967
7	6	10	10	9	9	1	0	893	0	44	0.0906	89 / 982
8	9	21	8	24	13	42	10	5	822	31	0.1655	163 / 985
9	7	6	4	6	40	5	0	39	11	885	0.1176	118 / 1,003
Total	968	1126	926	1000	1017	945	945	992	948	1056	0.1064	1,056 / 9,923

http://docs.h2o.ai/h2o/latest-stable/h2o-docs/performance-and-prediction.html

Accuracy and Error rate

1. Classification Accuracy, **Acc** - percent of correct classification.
2. Error rate,**Err** - percent of incorrect classification.

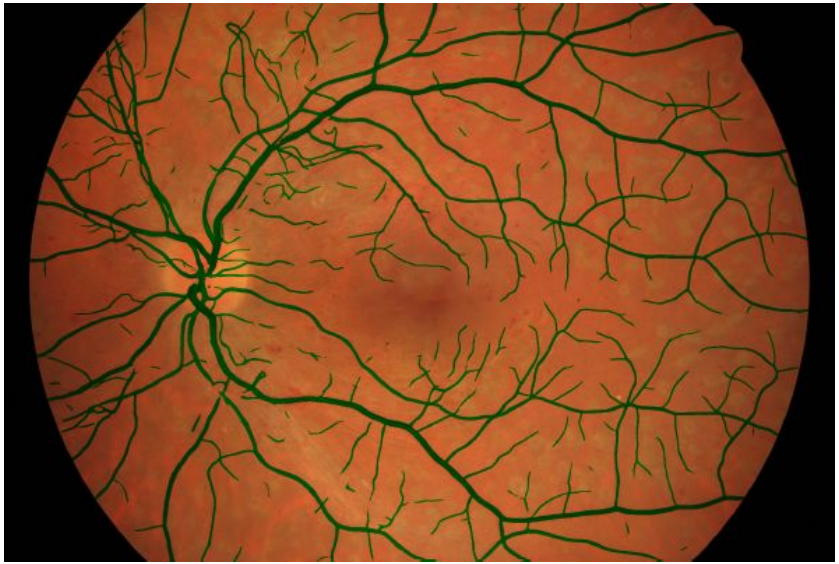
$$Acc = (100).\frac{TP+TN}{TP+TN+FP+FN}$$

$$Err = 1 - Acc$$

Pr, Re, F1

These measures (Pr, Re, F_1) don't care about number of true negatives (TN). This is because in many cases true negatives has no sense. Consider application of classification in image processing - segmentation of arteries and veins in retinal images by some classification approach.

There is around 10% of blood vessels (true positive) and the rest pixels belongs to non-vessels regions (false negative).Therefore, using measures with true negative, e.g. specificity (TN/(TN+FP)), would not be useful.



Precision

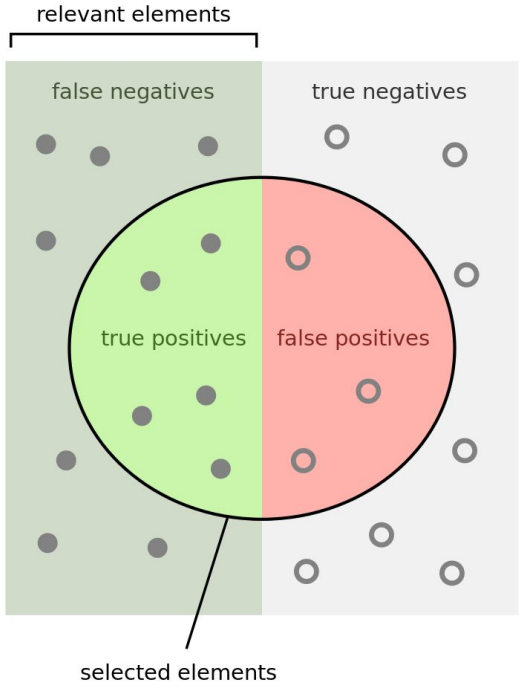
Positive prediction value (PPV) or precision:

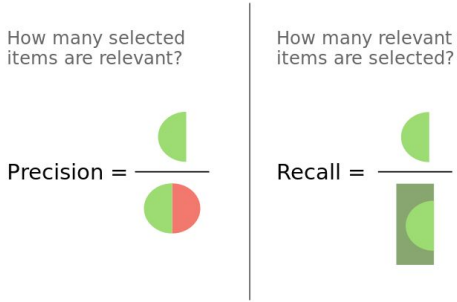
$$\textbf{\textit{Precision}} = TP / (TP + FP) =$$

$$\frac{\textit{Number of positive correctly classified samples}}{\textit{Total number of positive predicted samples}}$$

It tells us, what proportions of all the positive predictions are actually positive.

For example consider task of spam detection and the precision is 90%. This means that we correctly classified/detect 90% of relevant items - spam emails (10% of non-spam will be considered as a spam).





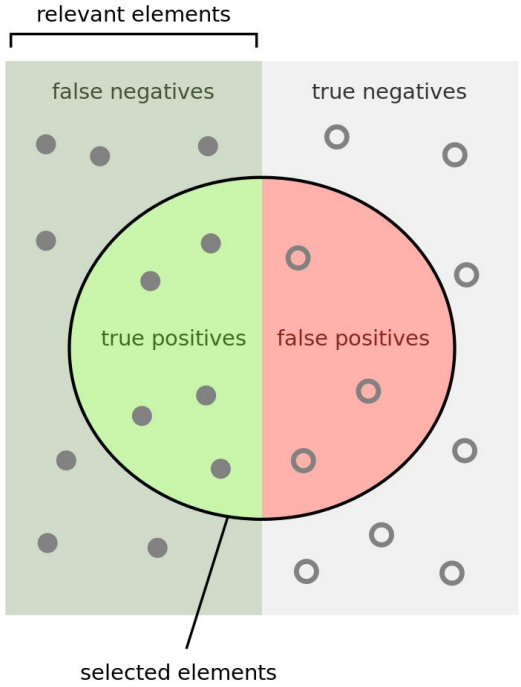
Recall

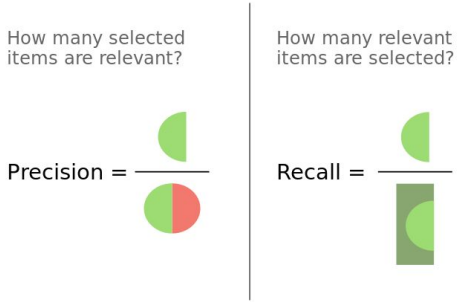
Recall known as Sensitivity, True positive rate (TPR) or hit rate:

$$\textbf{\textit{Recall}} = TP / (TP + FN) = \frac{\textit{Number of positive correctly classified samples}}{\textit{Total number of positive samples}}$$

It tells us, out of all the positive cases, how many positive cases does the model identify

For example consider task of spam detection and the recall is 90%. This means that we miss 10% of spam (10% of spam will be considered as a non-spam).





F_1 - measure

The F_1-measure (or F-measure/score) is a *harmonic mean* of precision and recall, i.e.:

$$F_1 = \frac{1}{\frac{\frac{1}{Pr}+\frac{1}{Re}}{2}} = \frac{2}{\frac{1}{Pr}+\frac{1}{Re}} = \frac{2\times Pr\times Re}{Pr+Re}$$

The harmonic mean is more intuitive than the arithmetic mean when computing a mean of ratios.

Suppose that you have a fingerprint recognition system and its precision and recall be 1.0 and 0.2, respectively. Intuitively, the total performance of the system should be very low because the system covers only 20% of the registered finger- prints, which means it is almost useless. The F_1-score is 1/3, whereas the arithmetic mean of 1 and 0.2 is 0.6.

F_β measure

The full definition of the F-measure is given as:

$$F_\beta = \frac{(\beta^2+1)\times Pr\times Re}{\beta^2 Pr+Re}$$

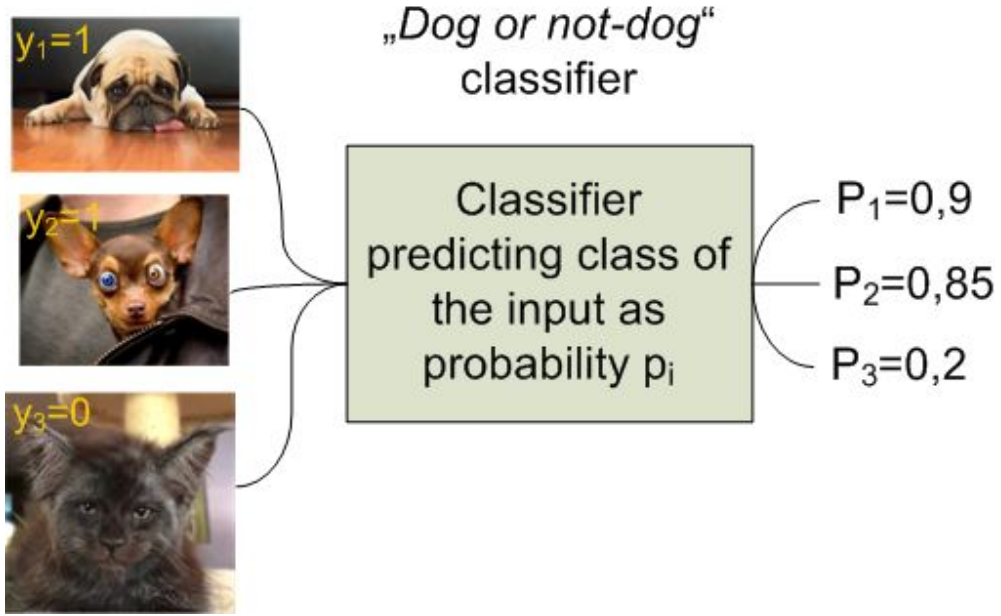
β is a parameter that controls a balance between *Pr* and *Re*. When *β = 1,* F_1 comes to be equivalent to the harmonic mean of *Pr* and *Re*. If *β > 1*, *F* becomes more *recall-oriented* and if *β < 1*, it becomes more *precision oriented*, e.g., $F_0 = Pr.$

The F_1-score is also known as the Sørensen–Dice coefficient or Dice similarity coefficient *or just* Dice coefficient.

Logarithmic Loss, Log Loss

Log loss measures the uncertainty of the probabilities of your model by comparing them to the true labels. It is used for two or more classes.

Consider that our classifier predict the input in term of probabilities - so we have labels (0 or 1), but classifier's output is a prediction. So how do we calculate an error of our model?



$$LogLoss = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log p_i + (1-y_i)\log(1-p_i))$$

Log Loss

Consider four cases (y_i denotes class (0 or 1) and p_i denotes predicted probability):

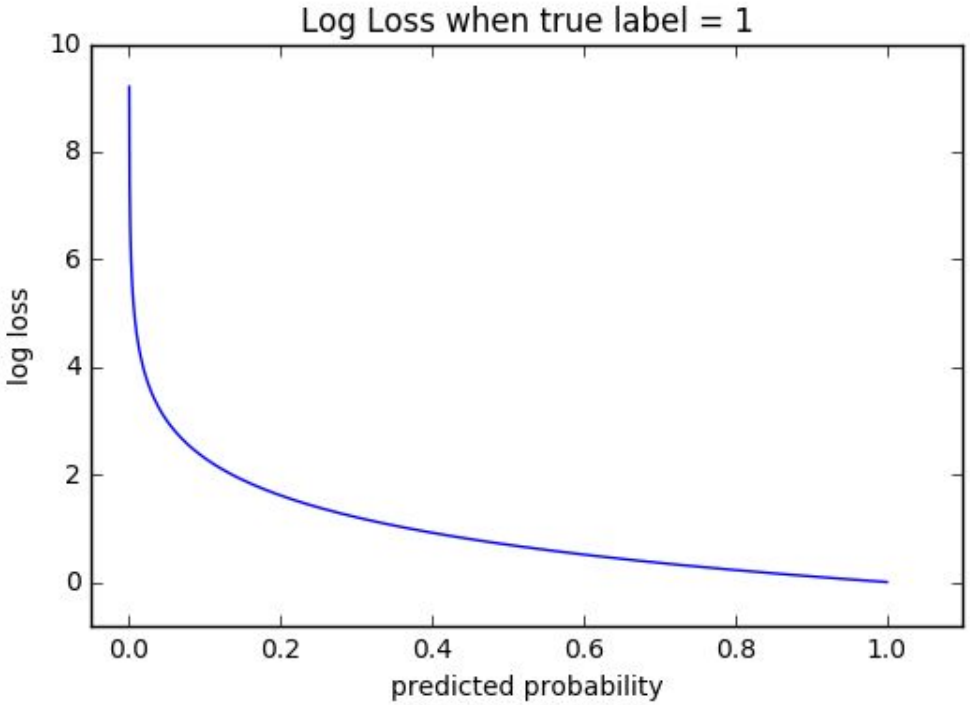
1) y_i = 1 and p_i = high (close to 1), this means that we are **predicting well**. We evaluate this case as $y_i . log(p_i)$, which will be **small**.
2) y_i = 1 and p_i = low (close to 0), this means that we are **predicting wrong**. We evaluate this case as $y_i . log(p_i)$, which will be **high**.
3) y_i = 0 and p_i = low (close to 0), this means that we are **predicting well**.We evaluate this case as $(1-y_i) . log(1 - p_i)$, which will be **small**.
4) y_i = 0 and p_i = high (close to 1), this means that we are **predicting wrong**.We evaluate this case as $(1-y_i) . log(1 - p_i)$, which will be **high**.

$$LogLoss = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log p_i + (1-y_i)\log(1-p_i))$$

Log Loss

The graph shows the range of possible loss values given a true observation (isDog, y_i=1). As the predicted probability approaches 1, log loss slowly decreases. However, as the predicted probability decreases, the log loss increases rapidly.

LogLoss can be used also for multiclass classification (One v. Rest).



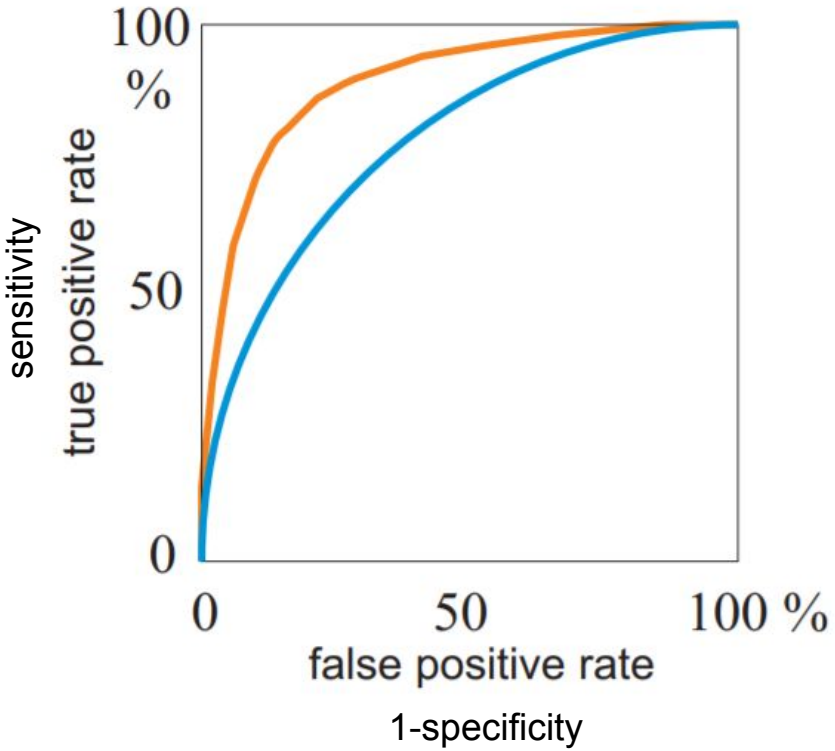
ROC analysis of classification

ROC analysis

ROC (Receiver Operating Characteristic/Curve) graphs are two-dimensional graphs in which ***TP rate (sensitivity)*** is plotted on the Y axis and ***FP rate (1-specificity)*** is plotted on the X axis. An ROC graph depicts relative tradeoffs between benefits (true positives) and costs (false positives).

Characterizes degree of overlap of classes.

Decision is based on a single threshold Θ (called also operating point).



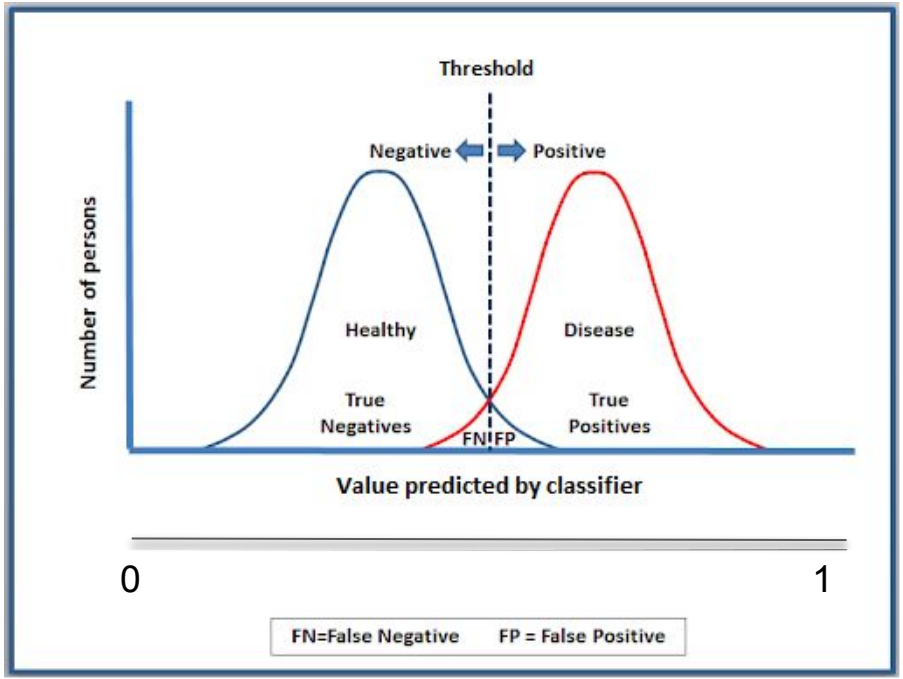
More information: Fawcet: Introduction to ROC analysis, https://www.sciencedirect.com/science/article/abs/pii/S016786550500303X

ROC curve

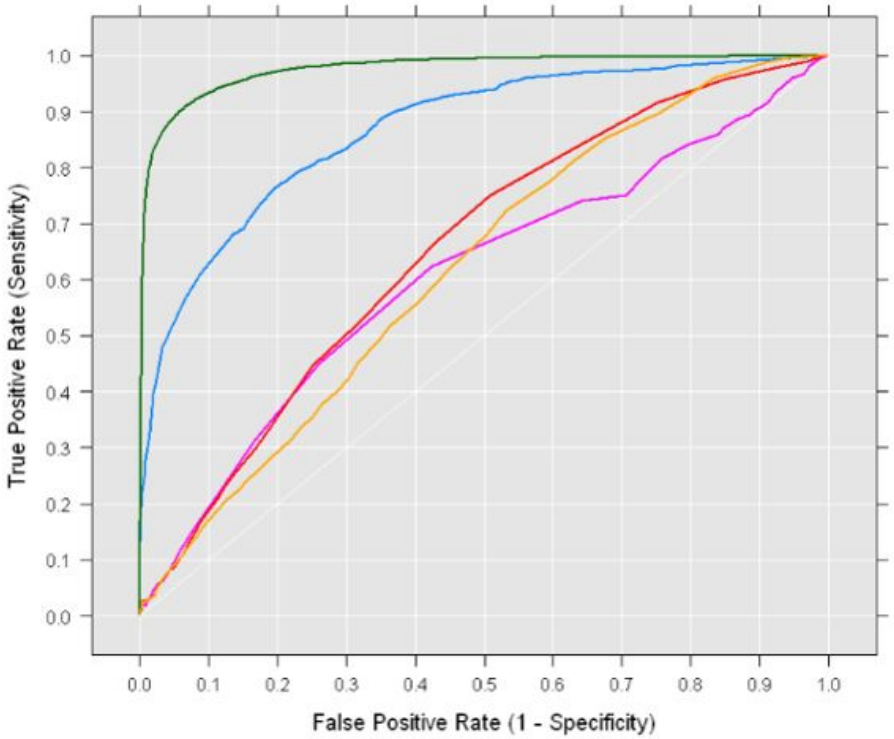
input - sample → Model (classifier) → **output** - some continuous variable (can be interpreted as a probability or likelihood) that describes “how much” the sample belongs to e.g. healthy class

Consider probability distribution function for two classes (healthy/disease, cat/dog...). These are represented by red and blue Gaussian curve in the image on the right. The y-axis represents the output of the classifiers, which is continuous and must be thresholded to classify input data into particular class.

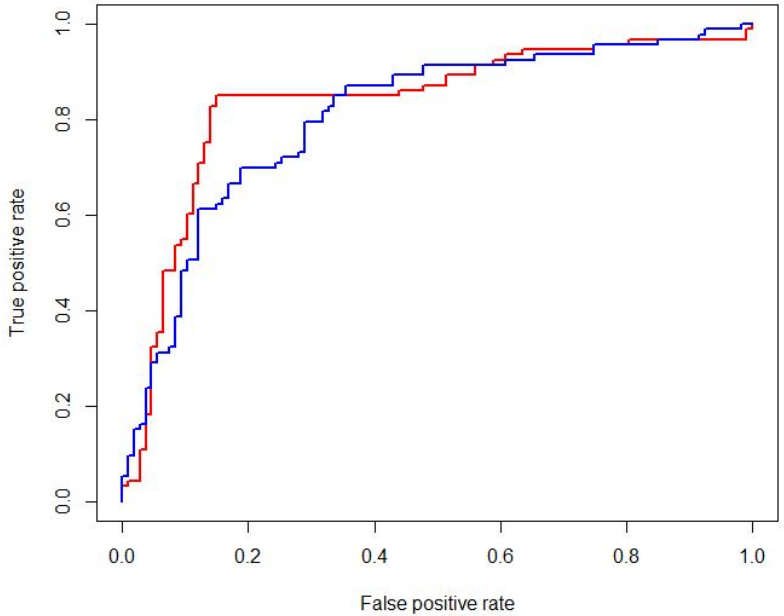
Moving the threshold will change the number of TP, TN, FN, FP and change the sensitivity and specificity value and therefore it will “generate” single point on the ROC curve.



ROC curve

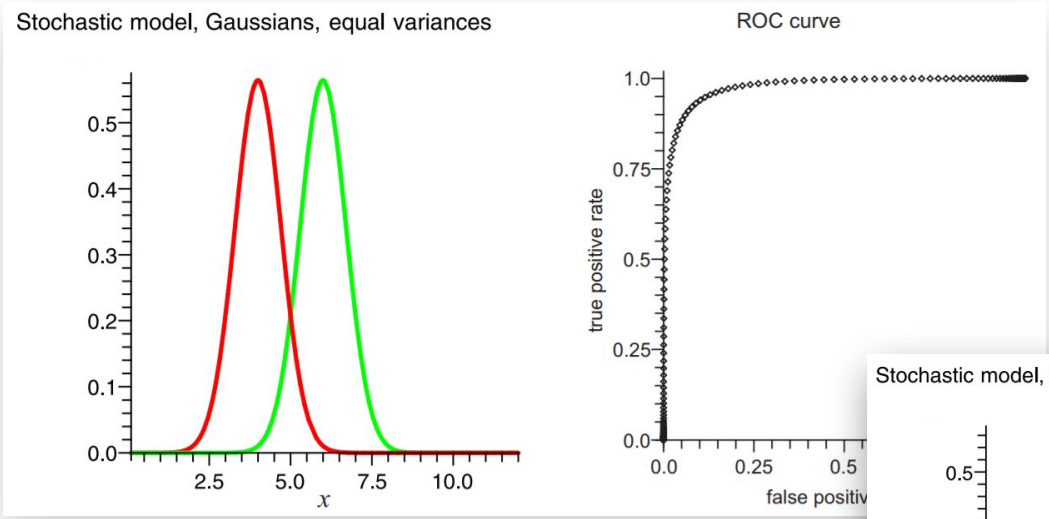


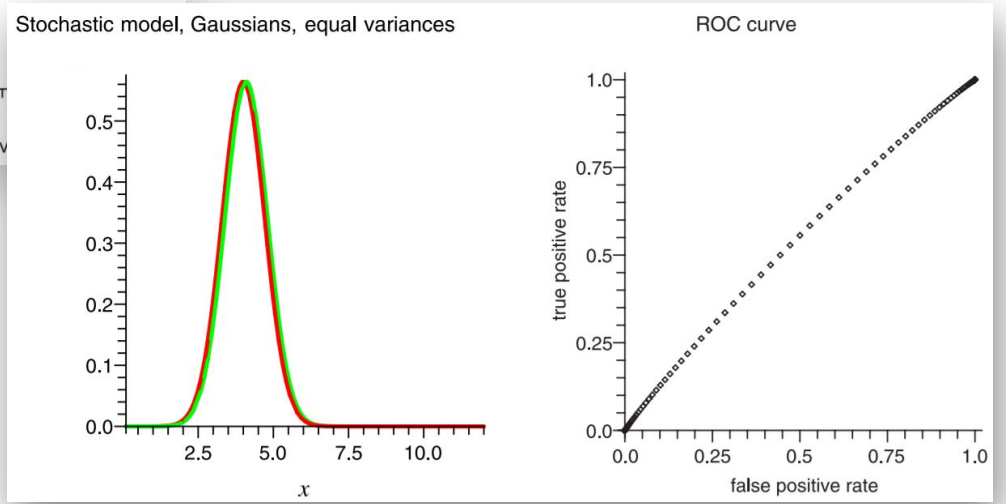
https://www.displayr.com/what-is-a-roc-curve-how-to-interpret-it/



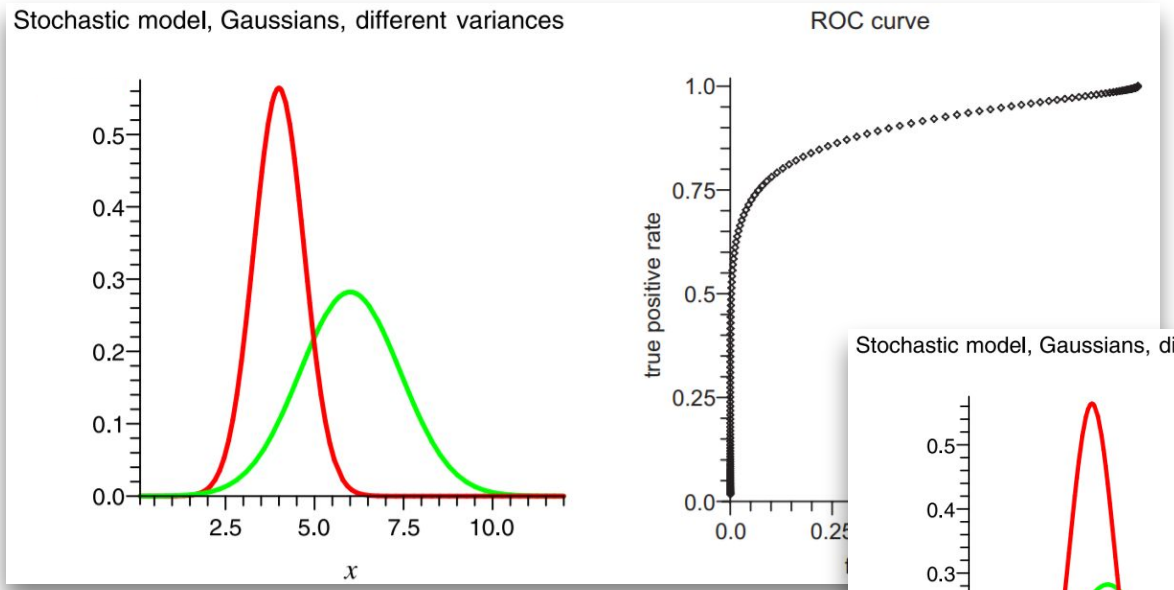
https://stackoverflow.com/questions/41833071/r-plot-multiple-different-coloured-roc-curves-using-rocr

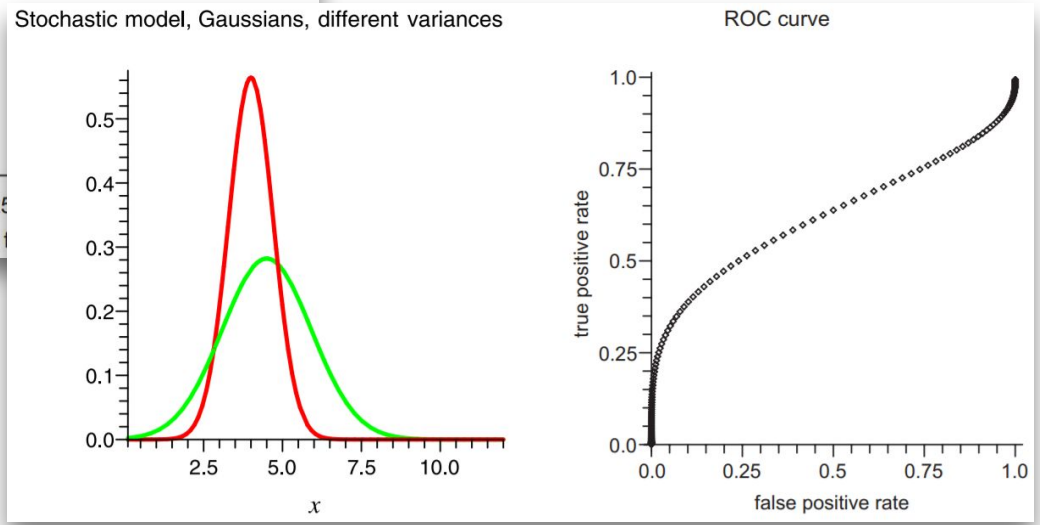
ROC - Gaussians, same variances, different overlap





ROC - Gaussians, different variances and overlap

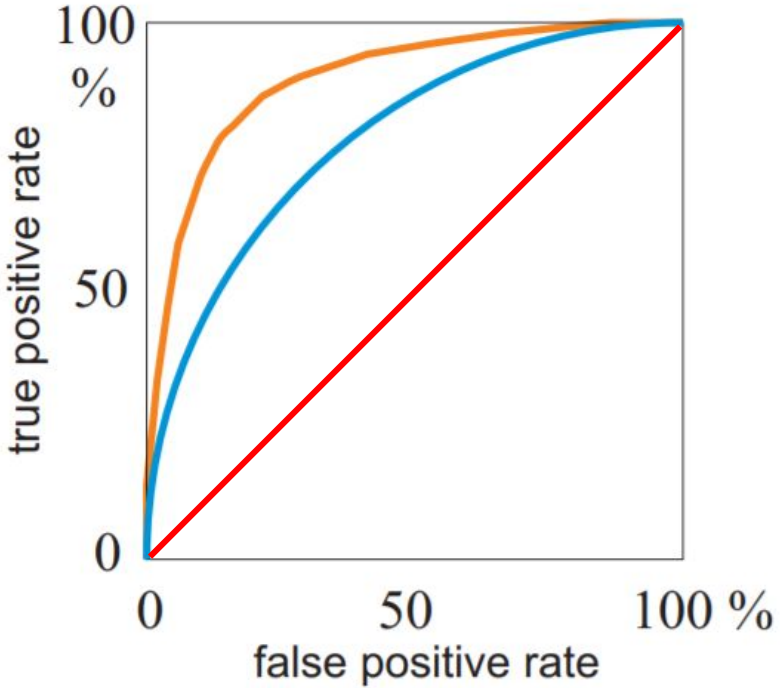




ROC - AUC

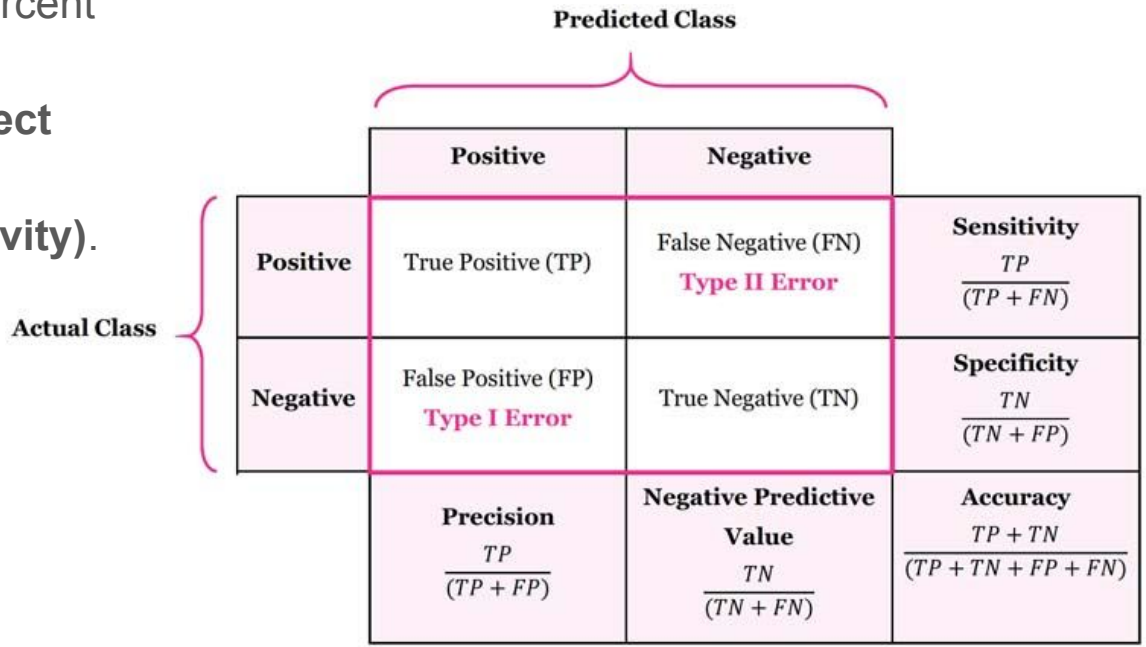
A classifier with the Red line is guessing the class randomly. Closer the ROC curve gets to top-left part of the chart, better the classifier is (yellow is better than blue).

Area under the curves (**AUC**) is the area below these ROC curves. Therefore, in other words, AUC is a great indicator of how well a classifier functions.



Conclusion - how to evaluate classifier?

1. **Classification Accuracy** - percent of **correct** clasification.
2. **Error rate** - percent of **incorrect** classification.
3. **Precision and recall (sensitivity)**.
4. **F1-score**
5. **LogLoss**
6. **Area Under ROC Curve**
7. **Confusion Matrix**



Tharwat, A. Classification assessment methods, https://doi.org/10.1016/j.aci.2018.08.003

Object (event) detection
precision-recall curve

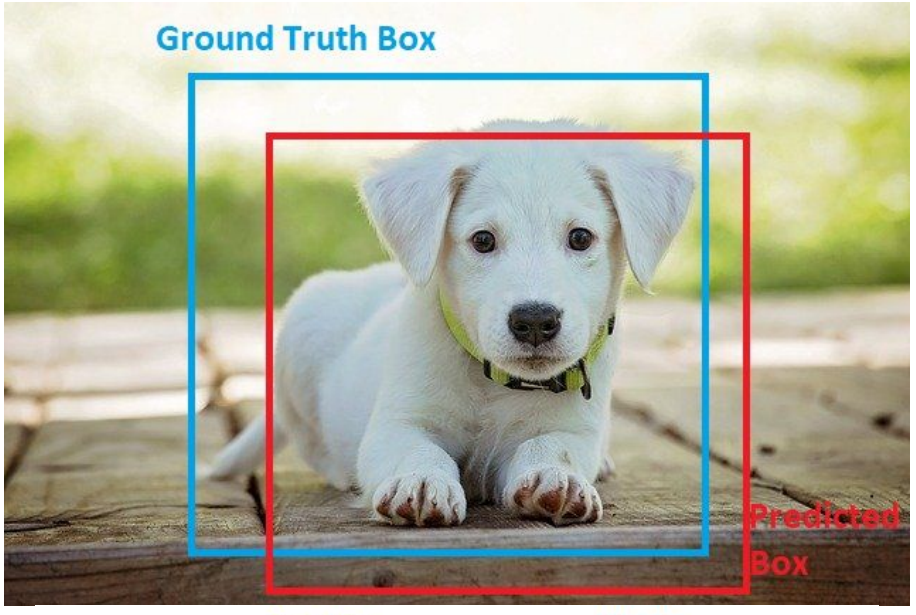
Object detection (not only in image data)

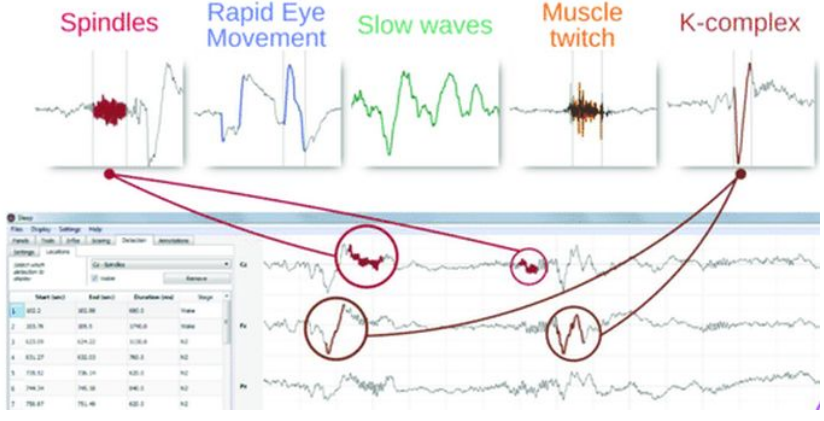
In object detection, we have to predict the bounding boxes around images.

After we predict the coordinates of the bounding box, how do we know how much accurate they are?

The Intersection Over Union can helps us to evaluate this task.

https://debuggercafe.com/wp-content/uploads/2020/08/video2_800_tomp4.mp4





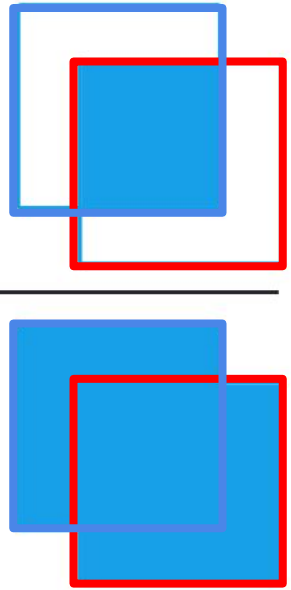
Intersection Over Union (IoU)

Generally, IoU gives the overlap between two bounding boxes. In object detection, it gives the overlap between the ground truth bounding box and the predicted bounding box.

But, how do we determine at which IoU value there is an object inside the predicted bounding box?

In most applications and competitions, an IoU of 0.5 is good enough.

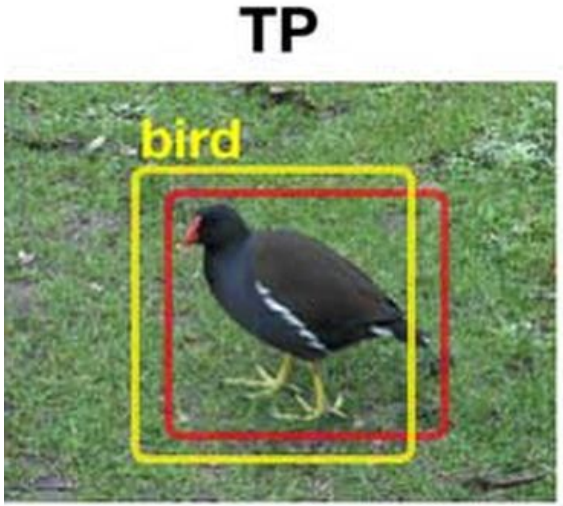
$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Intersection Over Union (IoU)

If IoU >= 0.5 then it is a true positive - this means that there is an object and we have detected it.

The detection algorithm correctly classifies the bird and draws correct bounding box around it.
This is an example of *true positive*.

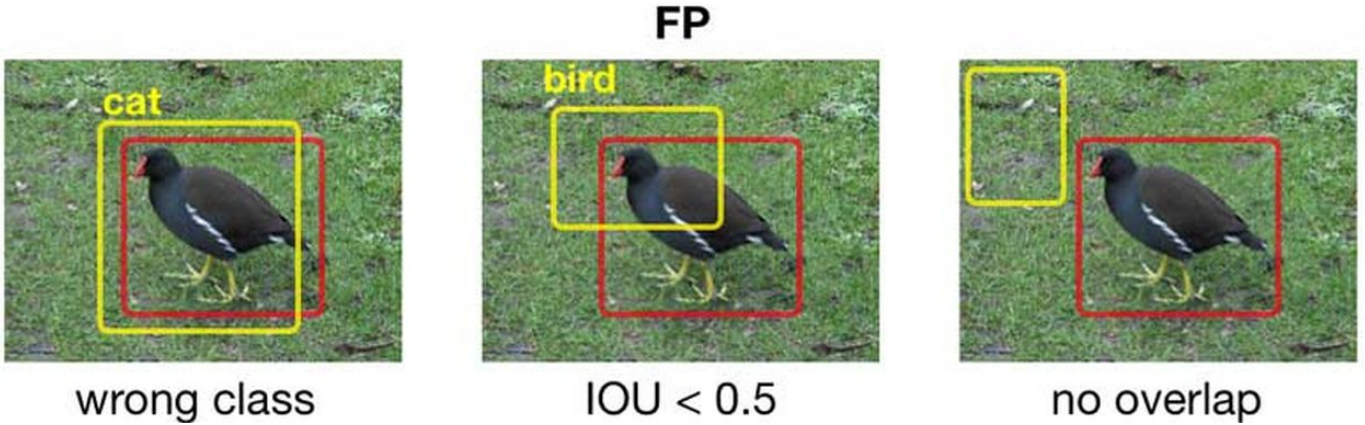


Red - Ground-truth
Yellow - Prediction

Intersection Over Union (IoU)

If IoU < 0.5 then it is a false positive - this means that we are drawing a bounding box around an object but classifying it as another object..

The algorithm detects animal, but it detects the bird as a cat. Then it detects only part of the bird (IoU<0.5) and also detect part of background as a bird.
These are cases of false positive.

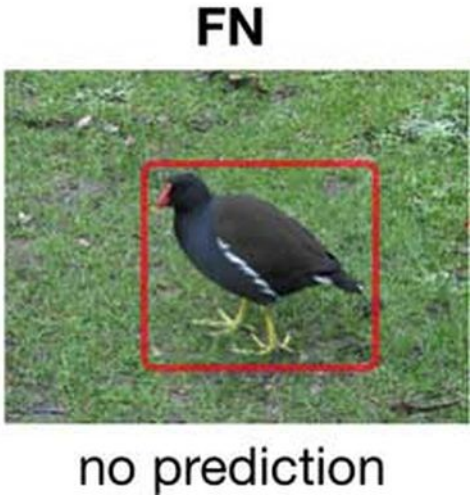


Red - Ground-truth
Yellow - Prediction

Intersection Over Union (IoU)

False negative - this is the case when the algorithm does not detect anything when it should be detecting.

The algorithm does not detect the bird in the image. *This is a case of false negative*.



Red - Ground-truth
Yellow - Prediction (missing here)

Precision - Recall curve

Precision-recall curve shows the tradeoff between the precision and recall values using the confidence of the predictor. This curve helps to select the best model's confidence to maximize both metrics. It is computed for specific IoU threshold.

Model's confidence - some measure, that tell us how the model is sure with the prediction ('It is class A with 78%.' or 'It is class A with 0.45.'). There is no standard definition of the term "confidence score" and you can find many different flavors of it depending on the models…

Some references to go through:

The PASCAL Visual Object Classes (VOC) Challenge paper

MS COCO vision dataset has become the standard to evaluate many of the object detection algorithms. The COCO competition provides the dataset for object detection, keypoint detection, segmentation, and also pose detection. Competition page, and the paper to get more details.

https://blog.zenggyu.com/en/post/2018-12-16/an-introduction-to-evaluation-metrics-for-object-detection/

Precision - Recall curve

Consider the following result of detection (for one class, e.g., *dog class*):

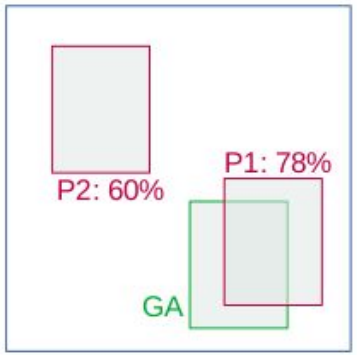


Image 1

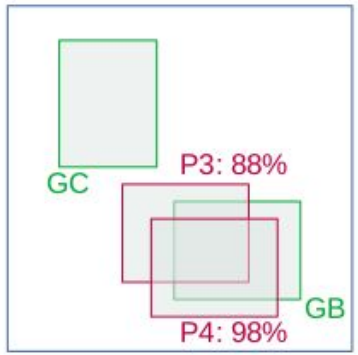


Image 2

Image	Detection	Ground Truth	Confidence	IoU
Image 1	P1	GA	78%	> 0.5
Image 1	P2	-	60%	< 0.5
Image 2	P3	GB	88%	> 0.5
Image 2	P4	GB	98%	> 0.5
Image 2	-	GC	-	-

We have two images, three ground-truth objects (green) and four prediction boxes (red). The number for each predicted box represent confidence value (a number given by detector).

https://kharshit.github.io/blog/2019/09/20/evaluation-metrics-for-object-detection-and-segmentation

Precision - Recall curve

Let's sort these values according to confidence:

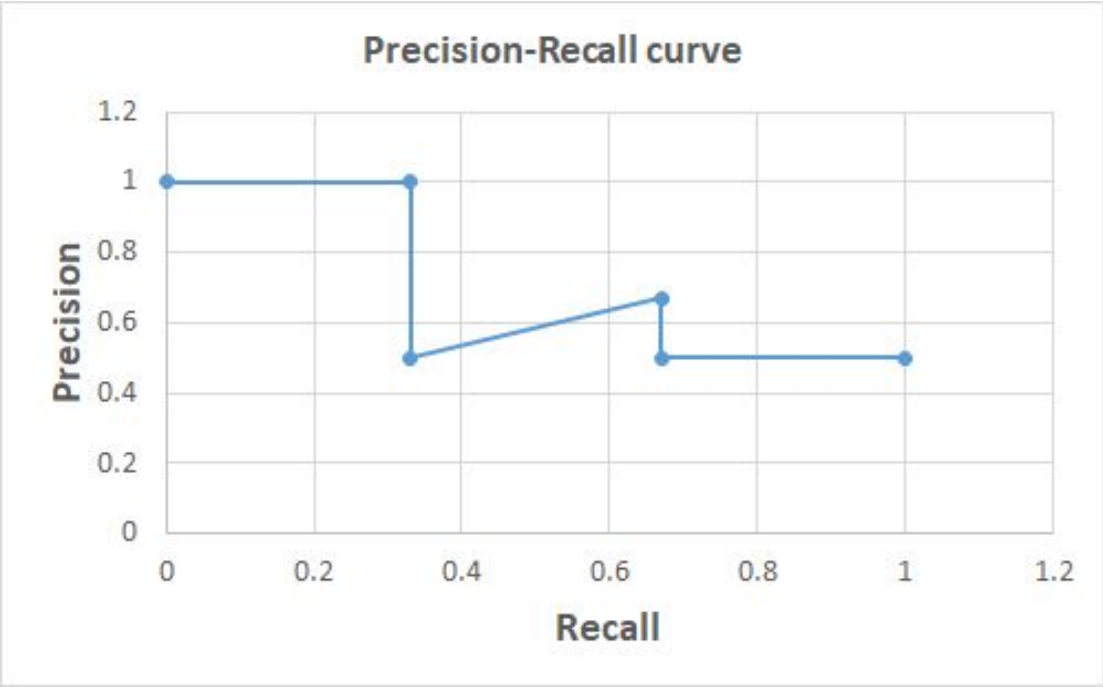
	Image	Detection	Confidence	IoU	Ground Truth	TP/FP	Accumulated TP	Accumulated FP	# detection by the model	# ground truth	Accumulated Precision	Accumulated Recall
Confidence ↓	Image 2	P4	98%	>0.5	GB	TP	1	0	1	3	1	0.33
	Image 2	P3	88%	>0.5	GB	FP	1	1	2	3	0.5	0.33
	Image 1	P1	78%	>0.5	GA	TP	2	1	3	3	0.67	0.67
	Image 1	P2	60%	<0.5		FP	2	2	4	3	0.5	0.67

After sorting, we can compute new Precision and Recall values according to Accumulated TP and Accumulated FP. This gives us the *accumulated* values depending on confidence. And finally, we can plot the Precision-Recall curve using these accumulated values.

Precision = TP / (TP + FP) = TP / #predictions

Recall = TP / (TP + FN) = TP / #ground truths

Average Precision



Confidence

Accumulated

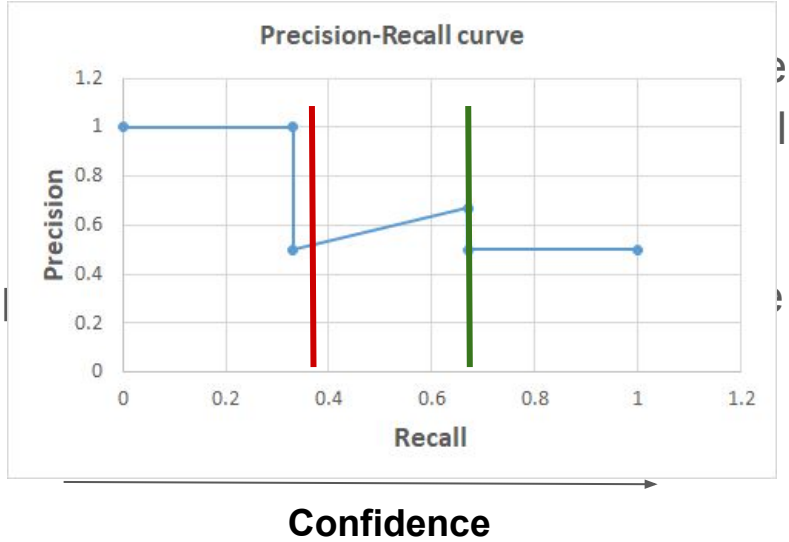
Precision	Recall
1	0.33
0.5	0.33
0.67	0.67
0.5	0.67

Precision - Recall

The precision-recall curve is typically decreasing in a zig-zag pattern (not monotonically). And it can be used to set our detection appropriately using the model confidence measure. It is obvious that it doesn't make sense to set the model such we will choose confidence threshold corresponding to green line rather than red line.

This is because for the green line, we can performance.

Generally, the model assessed using the Average Precision (AP).



Average Precision

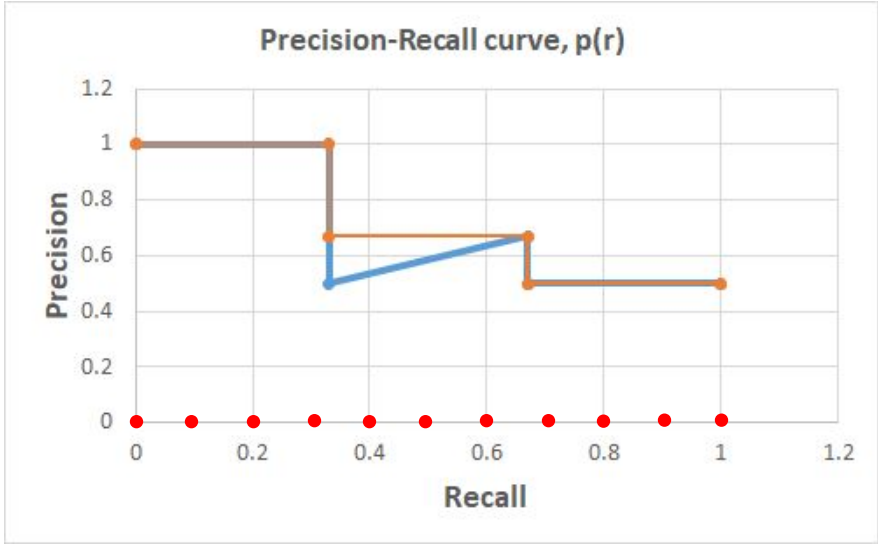
The AP summarizes the precision-recall curve in a single number and it is defined as the mean of precision values at a set of 11 equally spaced recall levels [0,0.1,…,1] (0 to 1 at step size of 0.1):

$$AP = \frac{1}{11} \sum_{r \in (0, 0.1, ..., 1)} p_{interp(r)}$$

The precision at each recall level r is interpolated by taking the maximum precision measured for a method for which the corresponding recall exceeds r:

$$p_{interp(r)} = \max_{\tilde{r}:\tilde{r} \geq r} p(r)$$

i.e. take the max precision value to the right at 11 equally spaced recall points [0, 0.1... 1], and take their mean to get AP.



Blue curve - original curve, *p(r)*
Orange curve - $p_{interp}(r)$

Mean Average Precision (mAP)

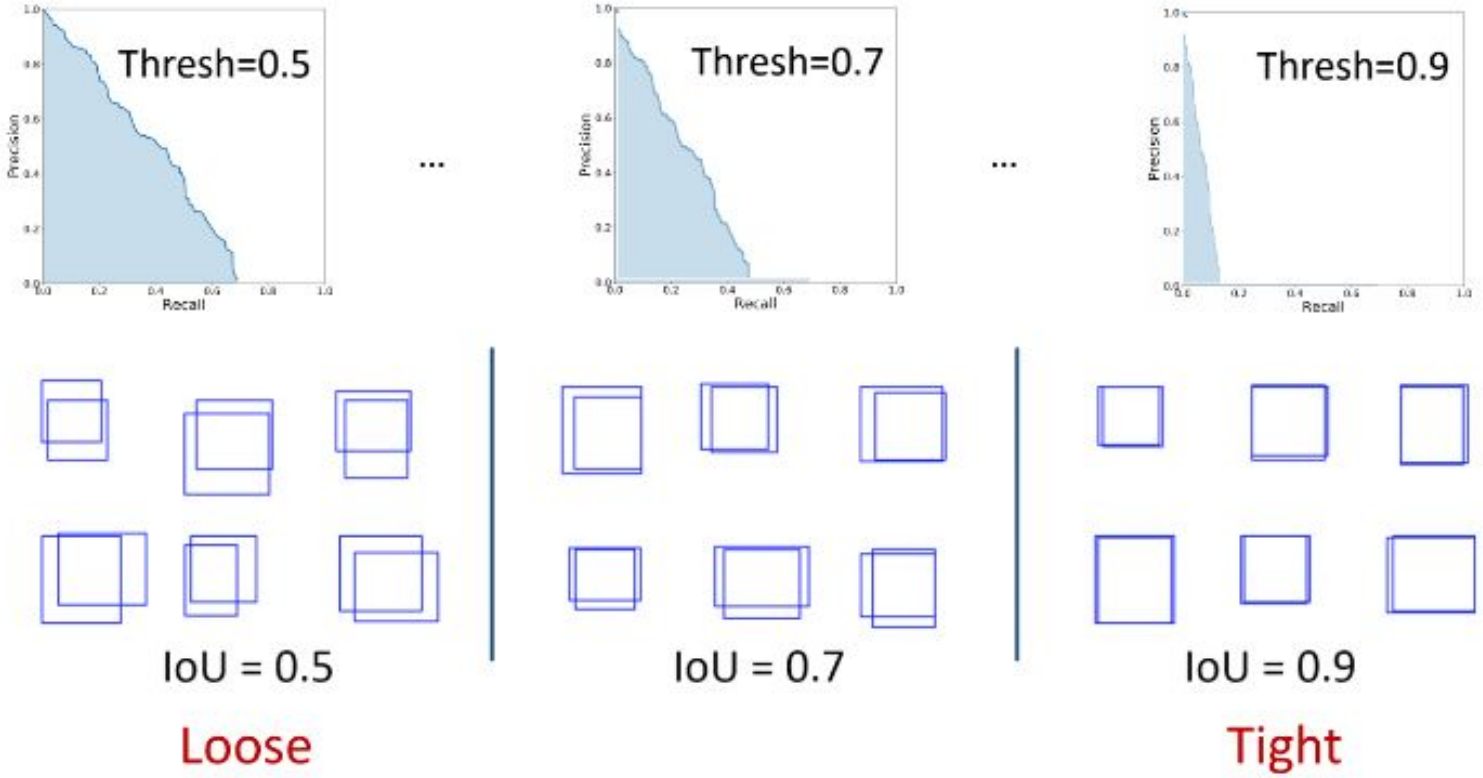
Class average: Now, we have AP per class (object category), mean Average Precision (mAP) is the averaged AP over all the object categories.

IoU average: However, the mAP also stands for the averaging over the different IoU, i.e. we use 10 IoU thresholds (0.50, 0.55, 0.6, … 0.95). The final value is:

$$mAP = \frac{mAP_{0.50} + mAP_{0.55} + ... + mAP_{0.95}}{10}$$

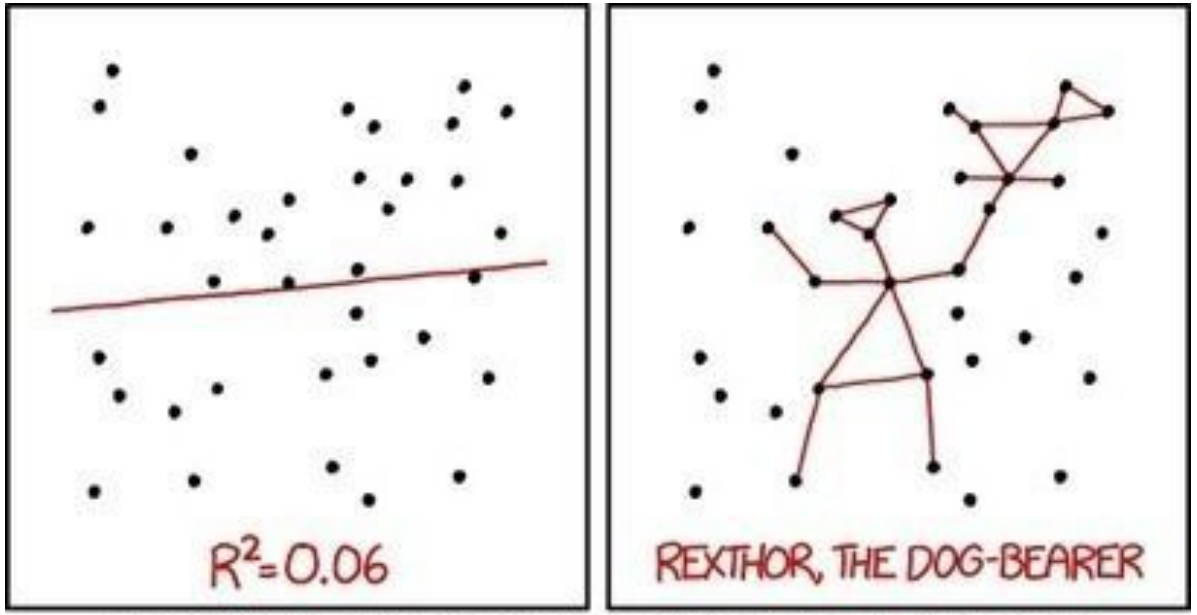
Thus the standard way for multiclass detection is to compute the mAP for these IoU thresholds and the compute the average over all classes.

Example - Precision-Recall curves for different IoU thresholds



https://kharshit.github.io/blog/2019/09/20/evaluation-metrics-for-object-detection-and-segmentation

Regression performance



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER TO GUESS THE DIRECTION OF THE CORRELATION FROM THE SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

Performance of regression

The **Mean Absolute Error (MAE)** - calculate the residual for every data point, taking only the absolute value:

$$MAE = \frac{1}{n}\sum \left| y - \hat{y} \right|$$

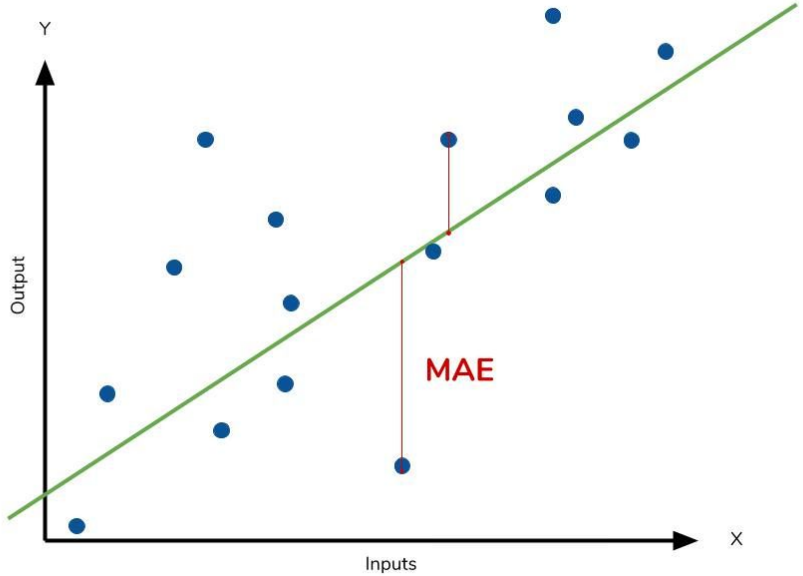
Divide by the total number of data points

Predicted output value

Actual output value

Sum of

The absolute value of the residual



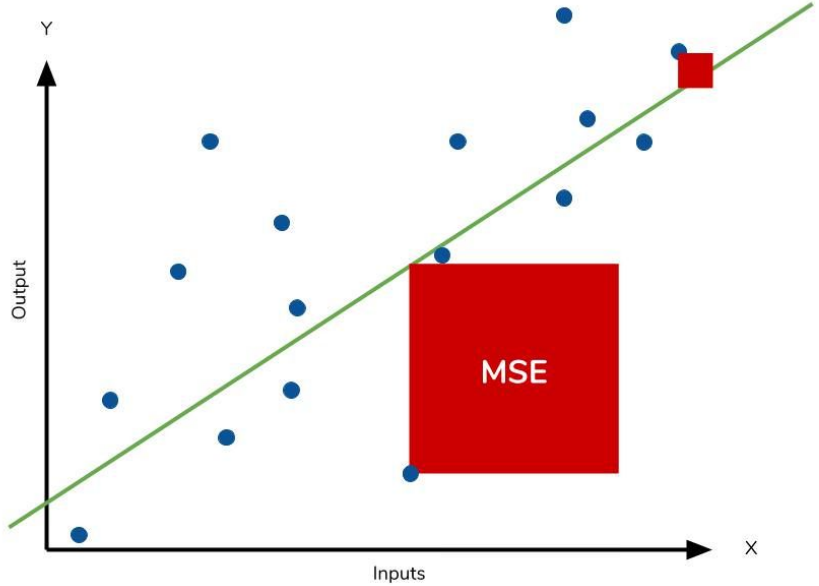
https://www.dataquest.io/blog/understanding-regression-error-metrics

Performance of regression

The **Mean Square Error (MSE)** is similar to MAE, but squares the difference before summing them all instead of using the absolute value.

$$MSE = \frac{1}{n}\Sigma\underbrace{\left(y - \hat{y}\right)}^{2}$$

The square of the difference between actual and predicted



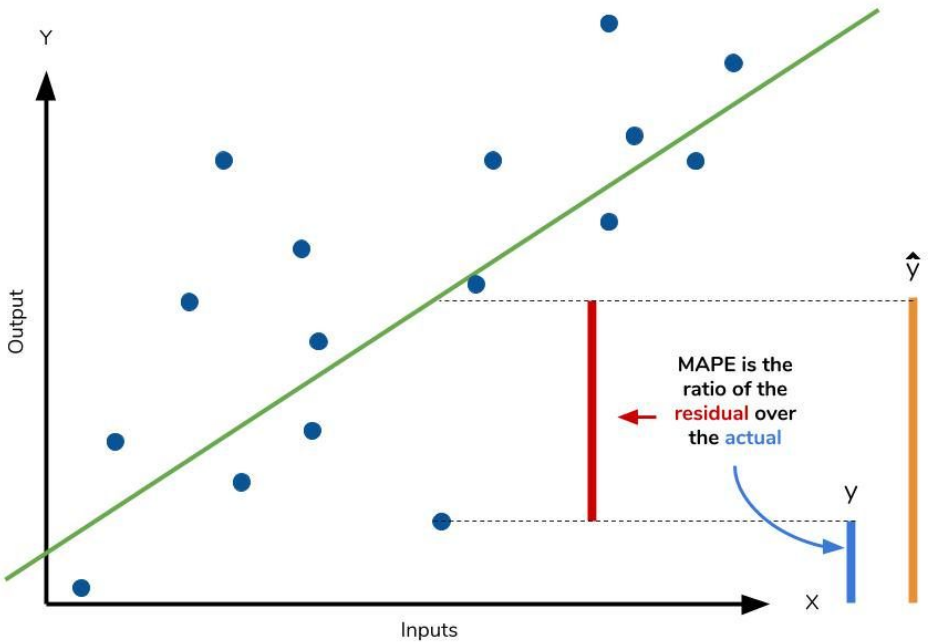
Performance of regression

The **Mean Absolute Percentage Error** (MAPE) is the percentage equivalent of MAE, except that the error is relative to give output value y:

$$MAPE = \frac{100\%}{n} \sum \left| \frac{\overbrace{y - \hat{y}}^{\text{The residual}}}{y} \right|$$

Multiplying by 100% converts to percentage

Each residual is scaled against the actual value



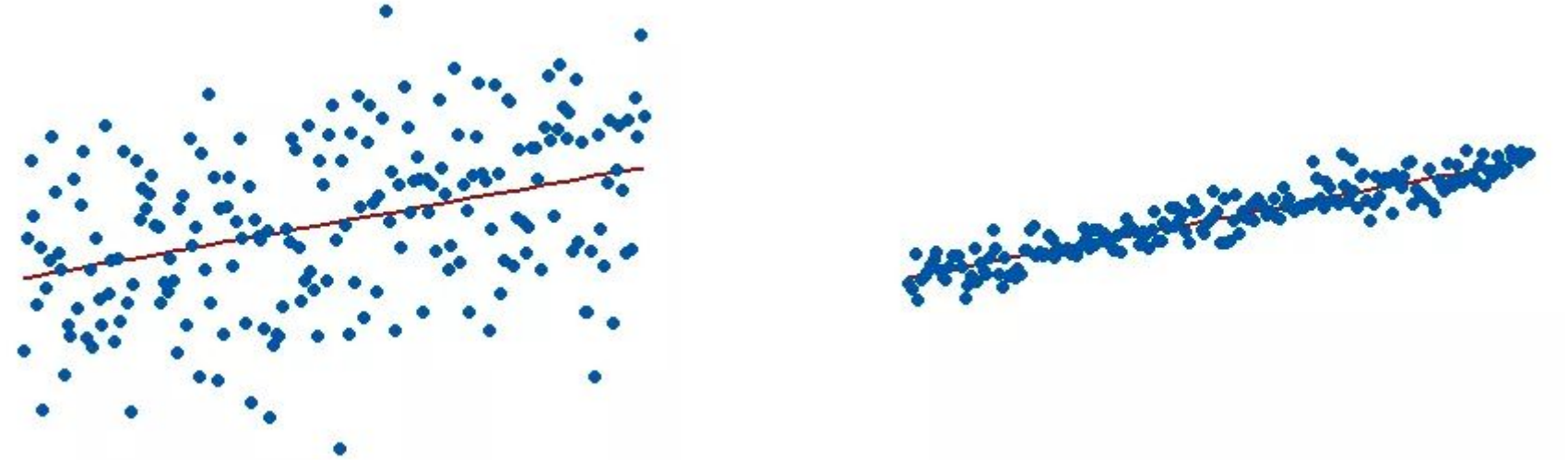
Performance of regression

Coefficient of determination, Goodness of fit, R^2 - quantifies the quality of regression and can be also interpreted as variation of the model with respect to total variation of the data.

We can compute the sum of squared error of regression model (SSE) as a sum of squared differences between the data (y) and prediction ($\hat{y}$). This will gives us some information about the quality of regression. But it will be depending on the data values. Therefore we will *normalize* it by variation (i.e. variance) of the data. The final R^2 value is then defined as:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y})^2}{\sum (y_i - \bar{y})^2}$$

R^2 value - example



R^2 for the regression model on the left is 15%, and for the model on the right it is 85%. When a regression model *covers* more variance, the data points are closer to the regression line/curve. In practice, a regression model with an R^2 of 100% is impossible. In that case, the fitted values equal the data values and, consequently, all of the observations fall exactly on the regression line.

R^2 value

R^2 value can be used for comparison of different models.

The values above 0.7 is usually desirable, between 0.8 - 0.9 is considered as good and above 0.9 as an excellent.

Conclusion - How to evaluate regressor?

1. Mean Absolute Error - MAE
2. Mean Squared Error - MSE
3. Mean Absolute Percentage Error - MAPE
4. R^2 value